

法律声明

□ 本课件包括：演示文稿，示例，代码，题库，视频和声音等，小象学院拥有完全知识产权的权利；只限于善意学习者在本课程使用，不得在课程范围外向任何第三方散播。任何其他人或机构不得盗版、复制、仿造其中的创意，我们将保留一切通过法律手段追究违反者的权利。

□ 课程详情请咨询

■ 微信公众号：小象

■ 新浪微博：ChinaHadoop



第十课 高级数据结构

林沐

内容概述

1.4道经典高级数据结构的相关题目

trie树(字典树)的基础知识

例1:实现trie树(medium)(trie树建立)

例2:添加与查找单词(medium)(trie树搜索)

并查集的基础知识

例3:朋友圈(medium)(并查集、图的搜索)

线段树的基础知识

例4:区域和的查询(medium)

2.详细讲解题目解题方法、代码实现

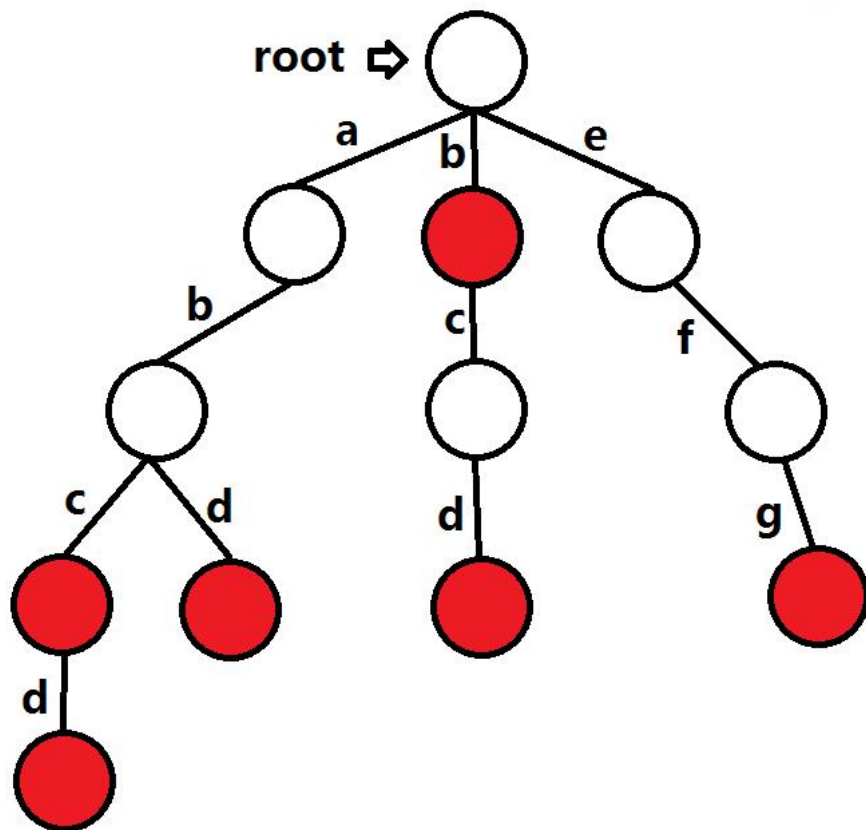
Trie树(字典树)概述

trie树，又称**字典树**或前缀树，是一种有序的、用于**统计、排序和存储**字符串的数据结构，它与二叉查找树不同，**关键字**不是直接保存在节点中，而是由节点在**树中的位置**决定。

一个节点的所有子孙都有**相同的前缀**，也就是这个节点对应的**字符串**，而根节点对应空字符串。一般情况下，不是所有的节点都有对应的值，只有**叶子节点**和**部分内部节点**所对应的键才有相关的值。

trie树的**最大优点**就是利用字符串的**公共前缀**来减少存储空间与查询时间，从而**最大限度地**减少无谓的字符串比较，是**非常高效**的字符串查找数据结构。

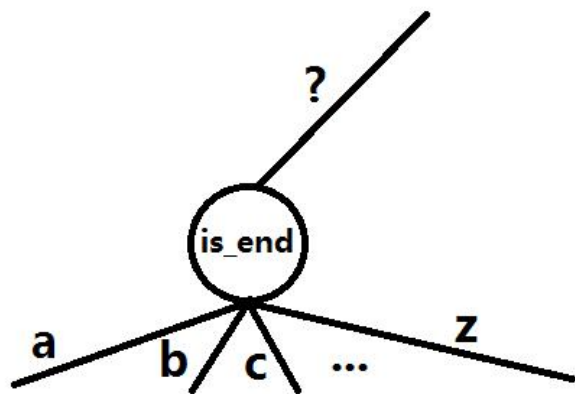
字符串: abc、abcd、abd、b、bcd、efg



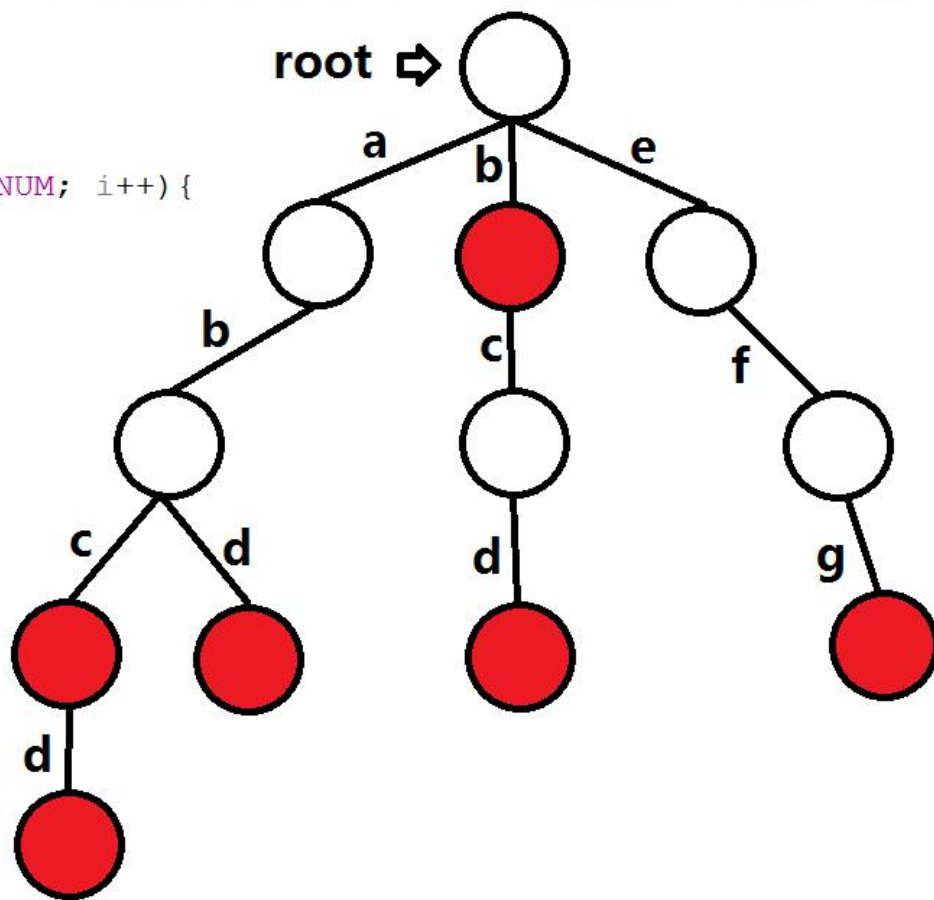
Trie树节点的表示

```
#define TRIE_MAX_CHAR_NUM 26
```

```
struct TrieNode{  
    TrieNode *child[TRIE_MAX_CHAR_NUM];  
    bool is_end;  
    TrieNode() : is_end(false){  
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){  
            child[i] = 0;  
        }  
    }  
};
```



字符串: abc、abcd、abd、b、bcd、efg



Trie树构造

```
int main(){
    TrieNode root;
    TrieNode n1;
    TrieNode n2;
    TrieNode n3;
    root.child['a'-'a'] = &n1;
    root.child['b'-'a'] = &n2;
    root.child['e'-'a'] = &n3;
    n2.is_end = true;

    TrieNode n4;
    TrieNode n5;
    TrieNode n6;
    n1.child['b'-'a'] = &n4;
    n2.child['c'-'a'] = &n5;
    n3.child['f'-'a'] = &n6;

    TrieNode n7;
    TrieNode n8;
    TrieNode n9;
    TrieNode n10;
    n4.child['c'-'a'] = &n7;
    n4.child['d'-'a'] = &n8;
    n5.child['d'-'a'] = &n9;
    n6.child['g'-'a'] = &n10;
    n7.is_end = true;
    n8.is_end = true;
    n9.is_end = true;
    n10.is_end = true;

    TrieNode n11;
    n7.child['d'-'a'] = &n11;
    n11.is_end = true;

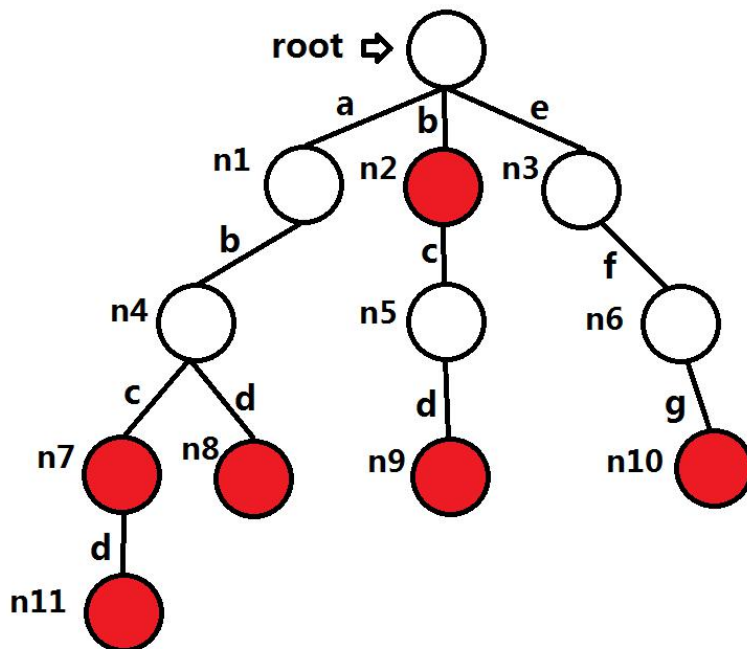
    preorder_trie(&root, 0);

    return 0;
}
```

```
#define TRIE_MAX_CHAR_NUM 26

struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```

字符串: abc、abcd、abd、b、bcd、efg



```
a
---b
-----c(end)
-----d(end)
-----d(end)
b(end)
---c
-----d(end)
e
---f
-----g(end)
请按任意键继续. . .
```

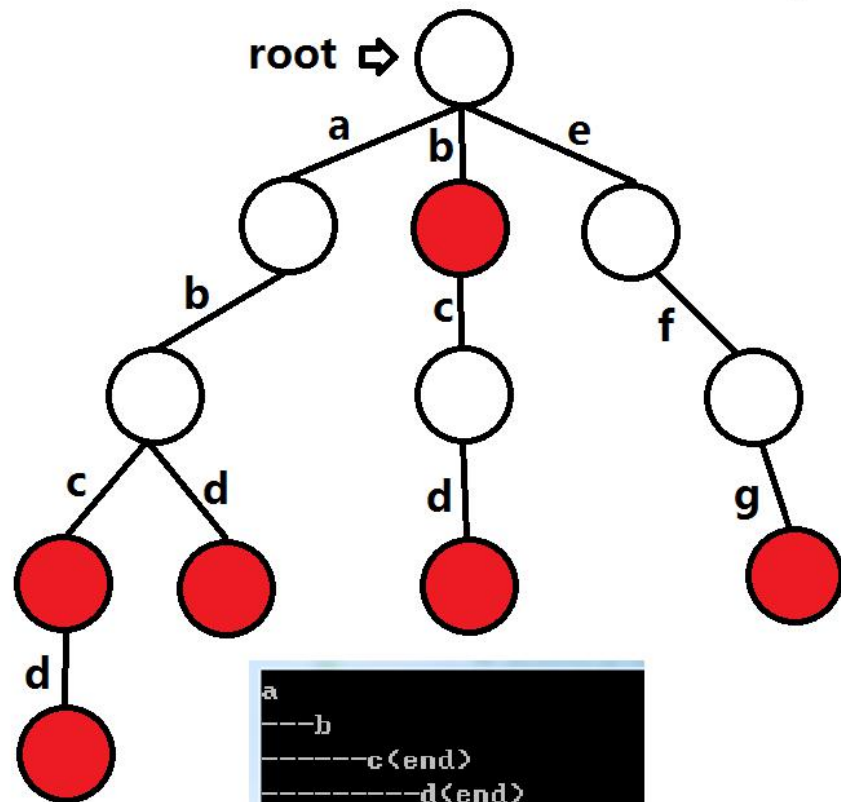
Trie树前序遍历，课堂练习

```
#define TRIE_MAX_CHAR_NUM 26
```

```
struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```

```
void preorder_trie(TrieNode *node, int layer){
    for (int i = 0; i < 1; i++){
        if (2){
            for (int j = 0; j < layer; j++){
                printf("---");
            }
            printf("%c", 3);
            if (4){
                printf("(end)");
            }
            printf("\n");
            preorder_trie(node->child[i], 5);
        }
    }
}
```

字符串: abc、abcd、abd、b、bcd、efg



```
a
---b
-----c(end)
-----d(end)
-----d(end)
b(end)
---c
-----d(end)
e
---f
-----g(end)
请按任意键继续. . .
```


Trie树前序遍历，实现

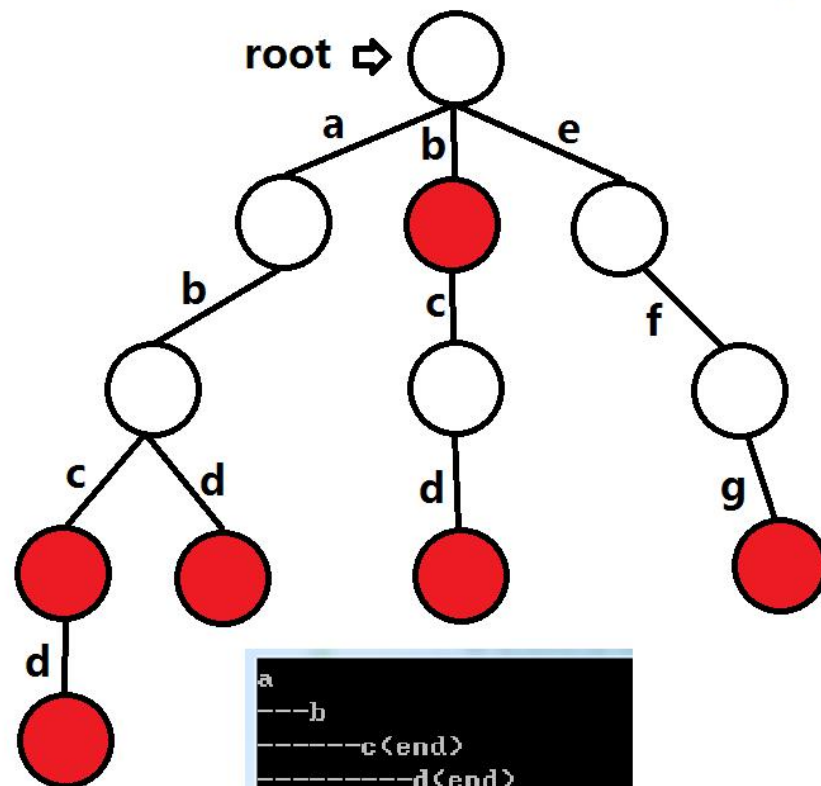
```
#define TRIE_MAX_CHAR_NUM 26

struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};

void preorder_trie(TrieNode *node, int layer){
    for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
        if (node->child[i]){
            for (int j = 0; j < layer; j++){
                printf("----");
            }
            printf("%c", i + 'a');

            if (node->child[i]->is_end){
                printf("(end)");
            }
            printf("\n");
            preorder_trie(node->child[i], layer + 1);
        }
    }
}
```

字符串: abc、abcd、abd、b、bcd、efg



```
a
---b
-----c(end)
-----d(end)
-----d(end)
b(end)
---c
-----d(end)
e
---f
-----g(end)
请按任意键继续. . .
```


Trie树获取全部单词

深度搜索trie树，对于正在搜索的节点node:

遍历该节点的26个孩子指针child[i]('a'-'z'), 如果**指针不空**:

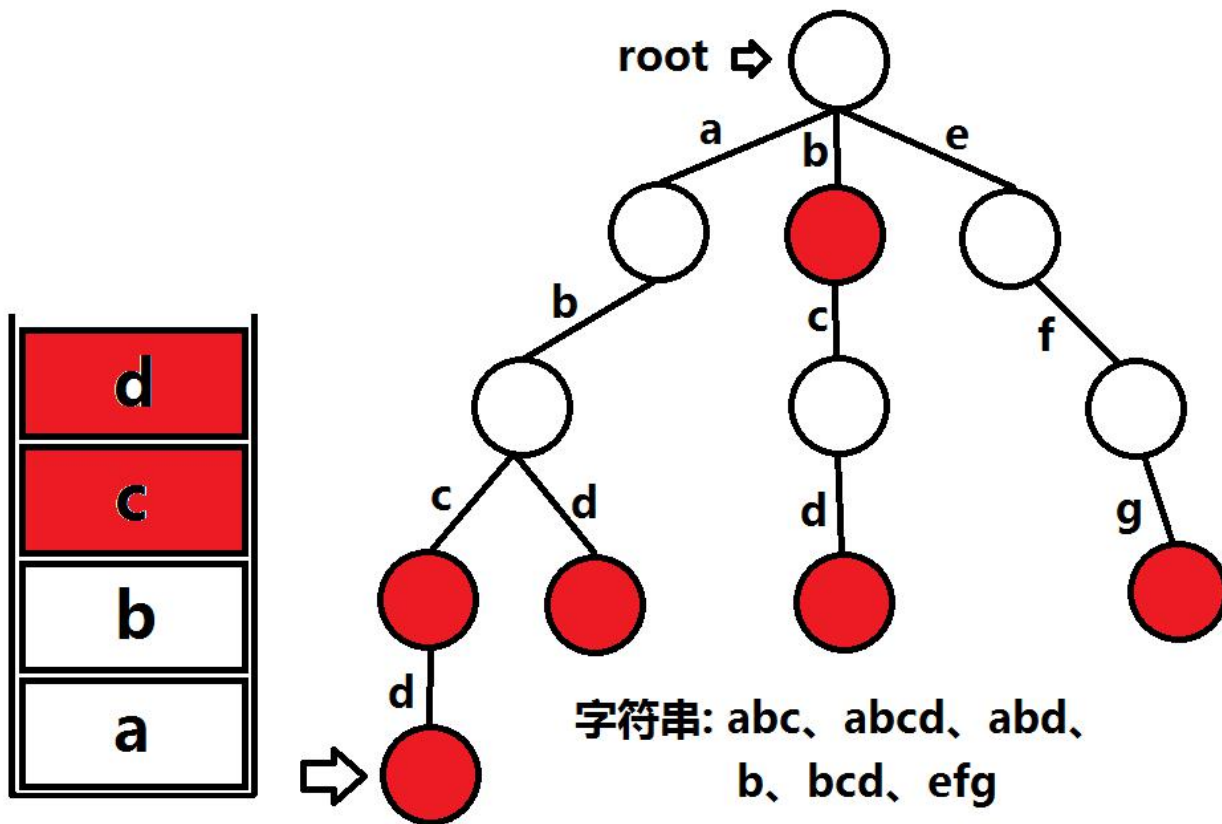
将该child[i]对应的字符(i+'a'), **push**进入栈中,

如果该孩子**指针标记**的is_end为真(说明这个位置是一个单词):

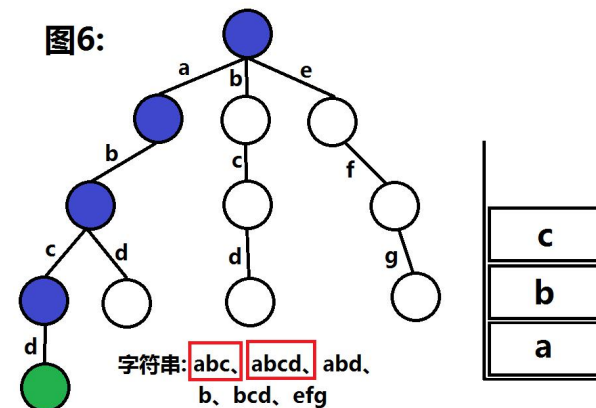
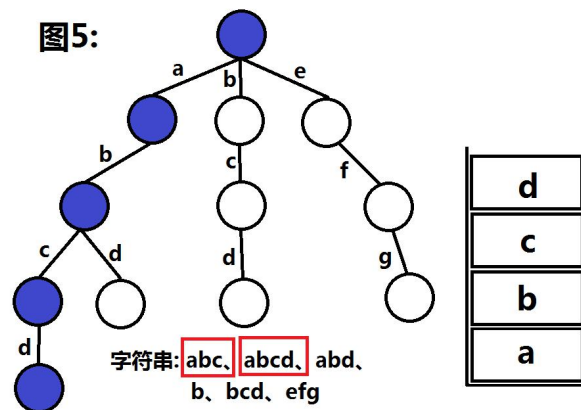
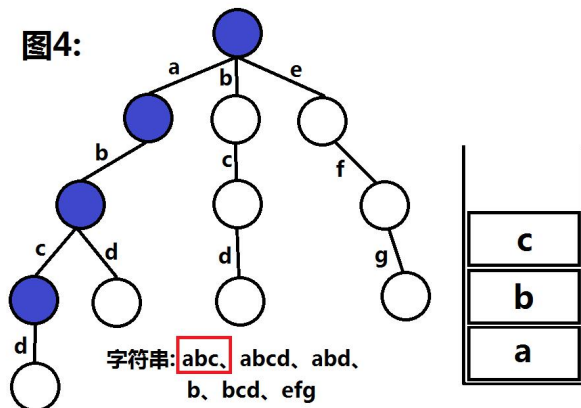
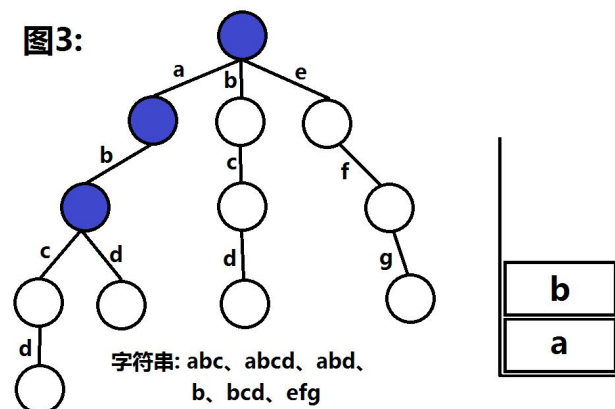
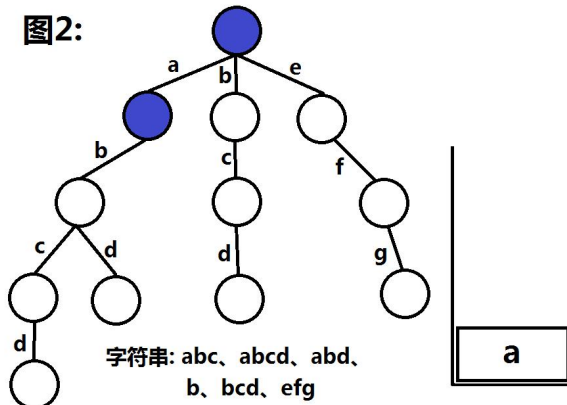
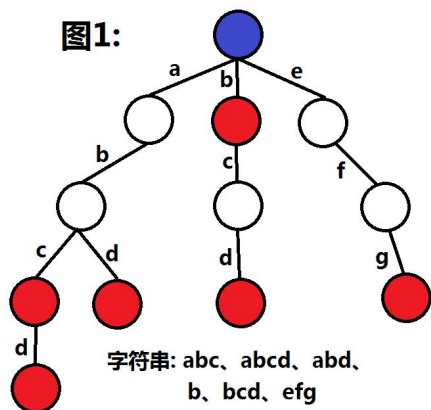
从栈底到栈顶对栈进行遍历，生成字符串，将它保存到结果数组中

深度搜索child[i]

弹出栈顶字符



Trie树获取全部单词



Trie树获取全部单词

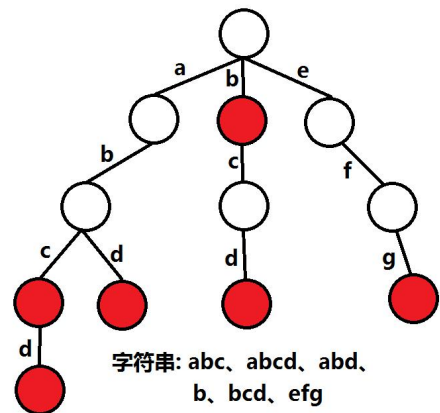


图7:

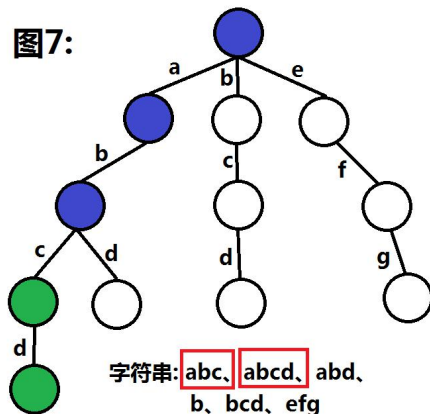


图8:

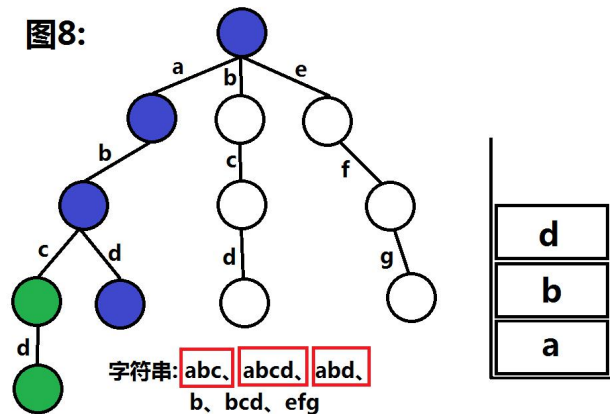


图9:

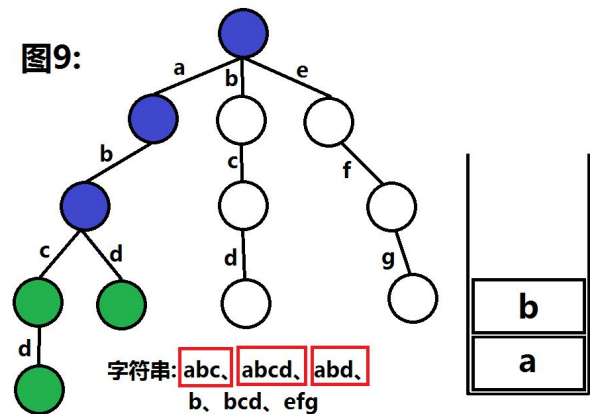


图10:

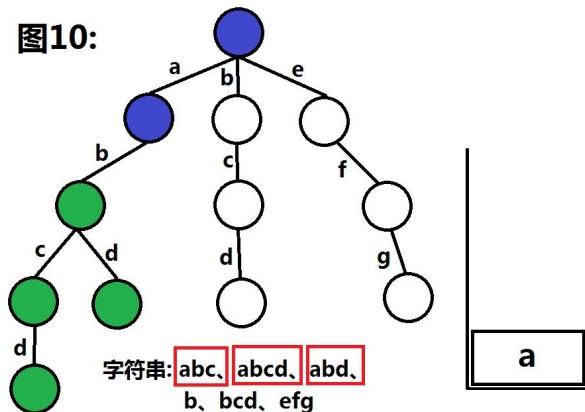
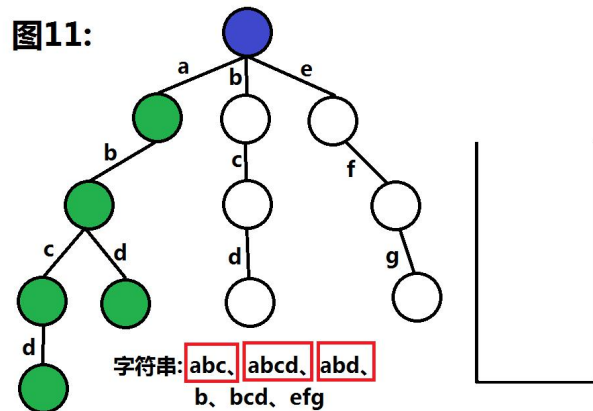


图11:



Trie树获取全部单词，课堂练习

```
#define TRIE_MAX_CHAR_NUM 26
```

```
struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```

//正在遍历的node //保存字符串的栈

```
void get_all_word_from_trie(TrieNode *node, std::string &word,
    std::vector<std::string> &word_list){
```

```
    for (int i = 0; i < 1; i++){
```

//最终结果

```
        if (2){
```

```
            word.push_back(3); //将字符入栈
```

```
            if (4){
```

```
                word_list.push_back(word);
```

```
            }
            get_all_word_from_trie(5, word, word_list);
```

```
            word.erase(word.length()-1, 1);
```

//弹出栈顶字符

```
abc
abcd
abd
b
bcd
efg
请按任意键继续. . .
```

3分钟时间填写代码，
有问题随时提出！

Trie树获取全部单词，实现

```
#define TRIE_MAX_CHAR_NUM 26
```

```
struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```

```
void get_all_word_from_trie(TrieNode *node, std::string &word,
    std::vector<std::string> &word_list){
    for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
        if (node->child[i]){
            word.push_back(i + 'a');
            if (node->child[i]->is_end){
                word_list.push_back(word);
            }
            get_all_word_from_trie(node->child[i], word, word_list);
            word.erase(word.length()-1, 1);
        }
    }
}
```

//正在遍历的node //保存字符串的栈

//最终结果

//将字符入栈

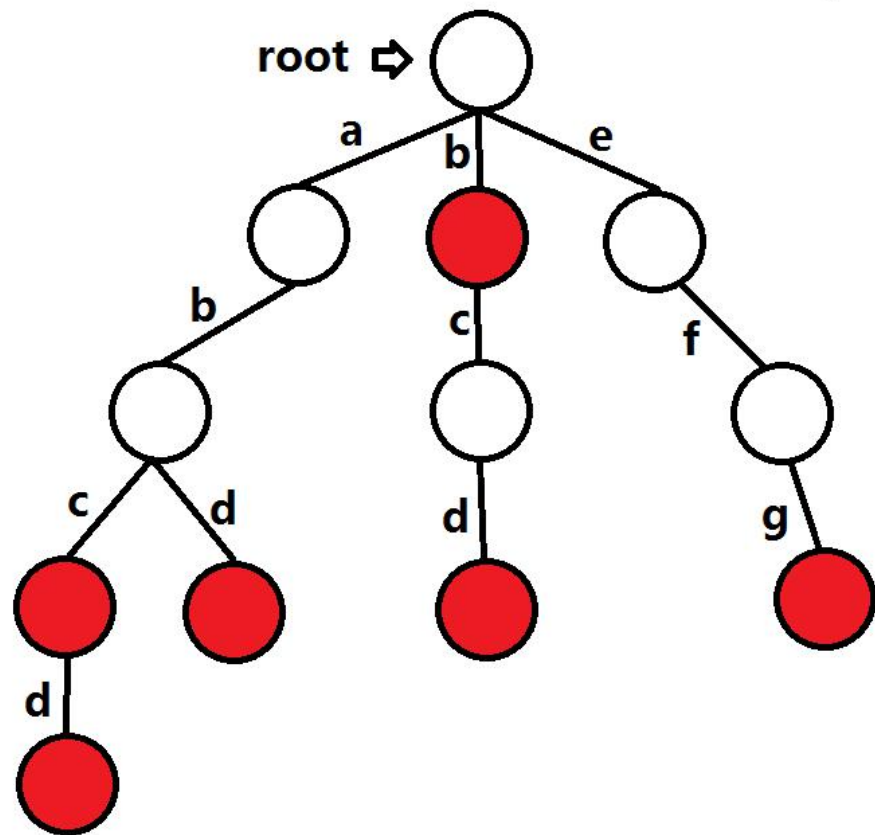
//弹出栈顶字符

```
abc
abcd
abd
b
bcd
efg
请按任意键继续. . .
```


Trie树的整体功能

```
class TrieTree{
public:
    TrieTree(){
    }
    ~TrieTree(){
        for (int i = 0; i < _node_vec.size(); i++){
            delete _node_vec[i];
        }
    }
    void insert(const char *word) {
        //将word插入至trie
    }
    bool search(const char *word){
        //搜索trie中是否存在word
    }
    bool startsWith(const char *prefix){
        //确实trie中是否有前缀为prefix的单词
    }
    TrieNode *root(){
        return &_root;
    }
private:
    TrieNode *new_node(){
        TrieNode *node = new TrieNode();
        _node_vec.push_back(node);
        return node;
    }
    std::vector<TrieNode *> _node_vec;
    TrieNode _root;
};
```

字符串: abc、abcd、abd、b、bcd、efg



Trie树的单词插入

使用ptr指针**指向**root；

逐个遍历待插入的字符串中的各个字符：

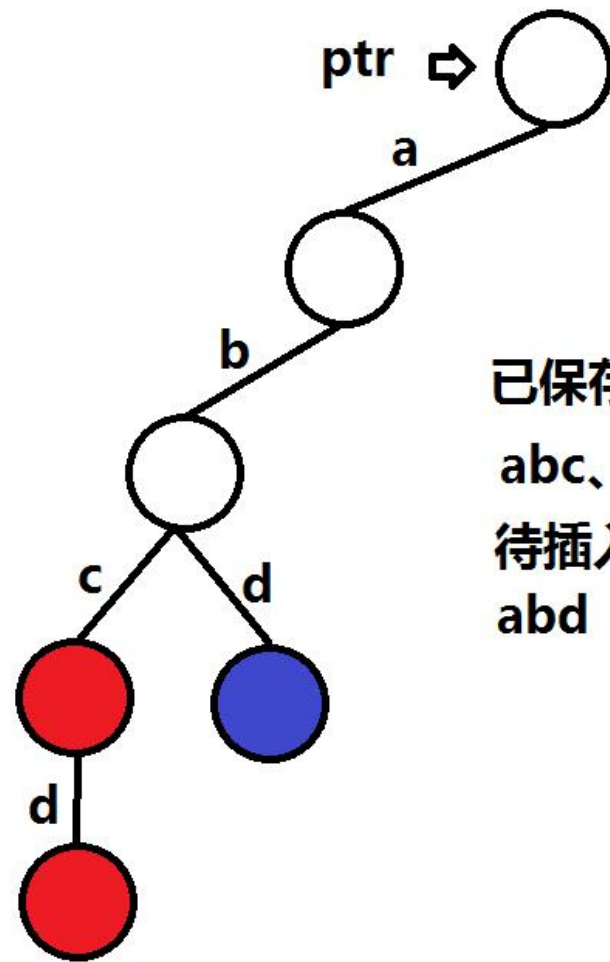
计算下标pos = 正在遍历字符 - 'a'；

如果ptr指向的节点的第pos个孩子为假：

创建该节点的第pos个孩子；

ptr**指向**该节点的第pos个孩子；

标记ptr指向的节点的is_end为true；



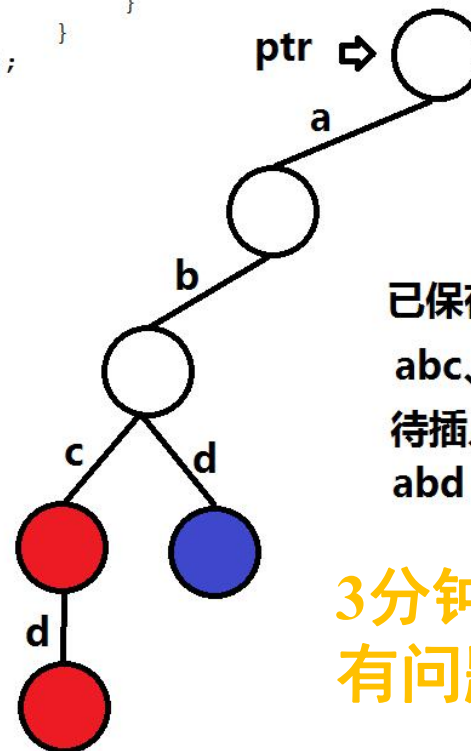
Trie树的单词插入:课堂练习

```
class TrieTree{
public:
    TrieTree(){
    }
    ~TrieTree(){
        for (int i = 0; i < _node_vec.size(); i++){
            delete _node_vec[i];
        }
    }
    void insert(const char *word) {
        TrieNode *ptr = 1
        while(*word){
            int pos = 2
            if ( 3 ){
                ptr->child[pos] = new_node();
            }
            ptr = 4
            word++;
        }
        5
    }
private:
    TrieNode *new_node() {
        TrieNode *node = new TrieNode();
        _node_vec.push_back(node);
        return node;
    }
    std::vector<TrieNode *> _node_vec;
    TrieNode _root;
};
```

```
#include <string>
#include <vector>

#define TRIE_MAX_CHAR_NUM 26

struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```



已保存字符串:

abc、abcd

待插入字符串:

abd

3分钟时间填写代码,
有问题随时提出!

Trie树的单词插入:实现

```
class TrieTree{
public:
    TrieTree(){
    }
    ~TrieTree(){
        for (int i = 0; i < _node_vec.size(); i++){
            delete _node_vec[i];
        }
    }
    void insert(const char *word) {
        TrieNode *ptr = &_root;

        while(*word){
            int pos = *word - 'a';

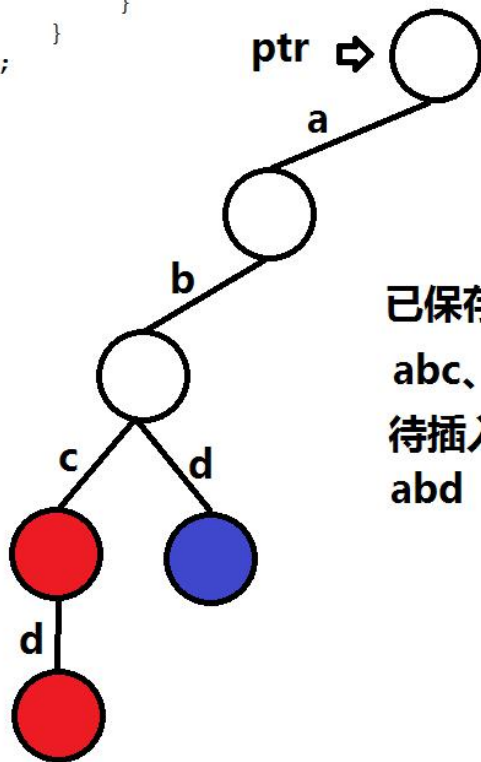
            if (!ptr->child[pos]){
                ptr->child[pos] = new_node();
            }
            ptr = ptr->child[pos];
            word++;

            ptr->is_end = true;
        }
    private:
        TrieNode *new_node(){
            TrieNode *node = new TrieNode();
            _node_vec.push_back(node);
            return node;
        }
        std::vector<TrieNode *> _node_vec;
        TrieNode _root;
};
```

```
#include <string>
#include <vector>

#define TRIE_MAX_CHAR_NUM 26

struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```



已保存字符串:
abc、abcd
待插入字符串:
abd

Trie树的单词搜索

使用ptr指针**指向**root；

逐个遍历待搜索的字符串中的**各个字符**：

计算**下标pos** = **正在遍历字符** - 'a'；

如果ptr指向的节点的**第pos个孩子**为假：

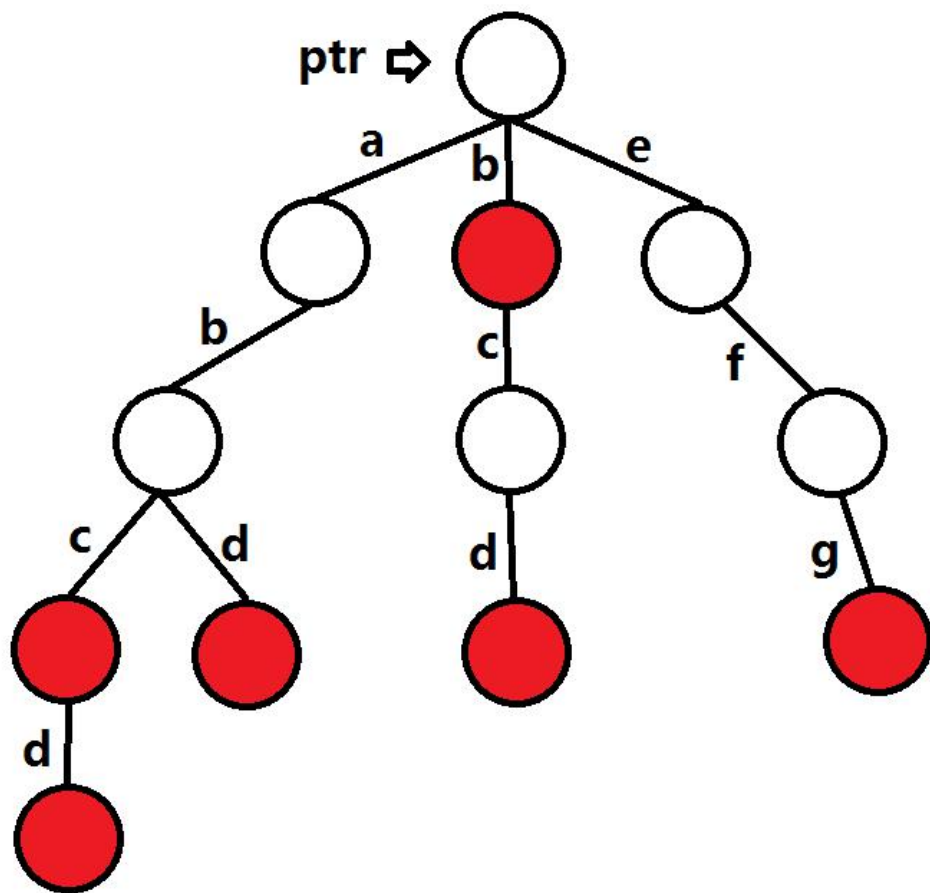
返回**假**；

ptr**指向**该节点的**第pos个孩子**；

返回ptr指向的节点的is_end

如搜索abc、ab、abe

字符串: abc、abcd、abd、b、bcd、efg



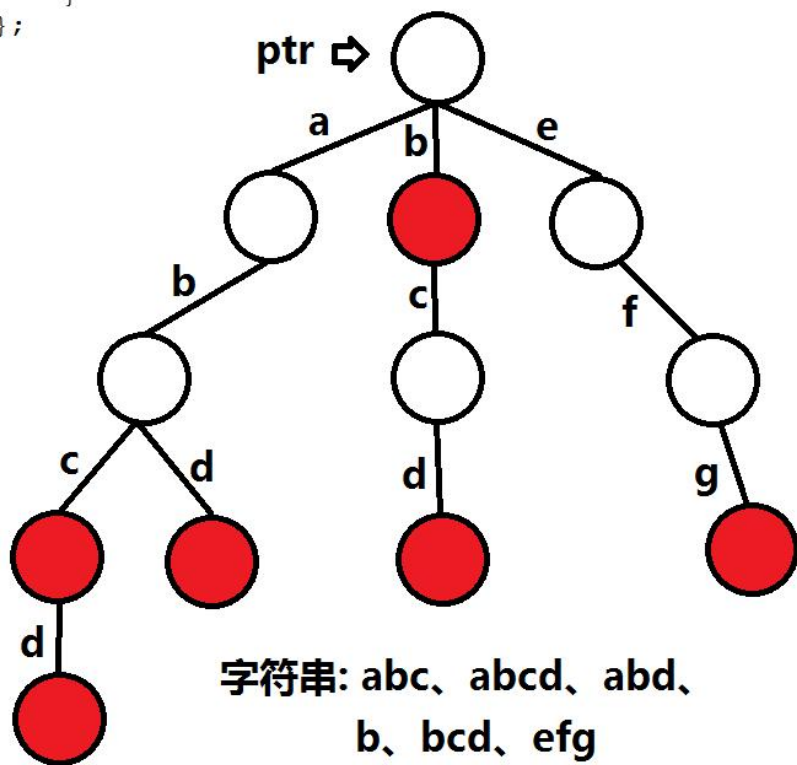
Trie树的单词搜索:课堂练习

```
class TrieTree{
public:
    TrieTree(){
    }
    ~TrieTree(){
        for (int i = 0; i < _node_vec.size(); i++){
            delete _node_vec[i];
        }
    }
    bool search(const char *word){
        TrieNode *ptr = 1
        while(*word){
            int pos = 2
            if (!ptr->child[pos]){
                3
            }
            ptr = 4
            word++;
        }
        5
    }
private:
    TrieNode *new_node(){
        TrieNode *node = new TrieNode();
        _node_vec.push_back(node);
        return node;
    }
    std::vector<TrieNode *> _node_vec;
    TrieNode _root;
};
```

```
#include <string>
#include <vector>

#define TRIE_MAX_CHAR_NUM 26

struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```



Trie树的单词搜索:实现

```
class TrieTree{
public:
    TrieTree(){}
    ~TrieTree(){
        for (int i = 0; i < _node_vec.size(); i++){
            delete _node_vec[i];
        }
    }
    bool search(const char *word){
        TrieNode *ptr = &_root;

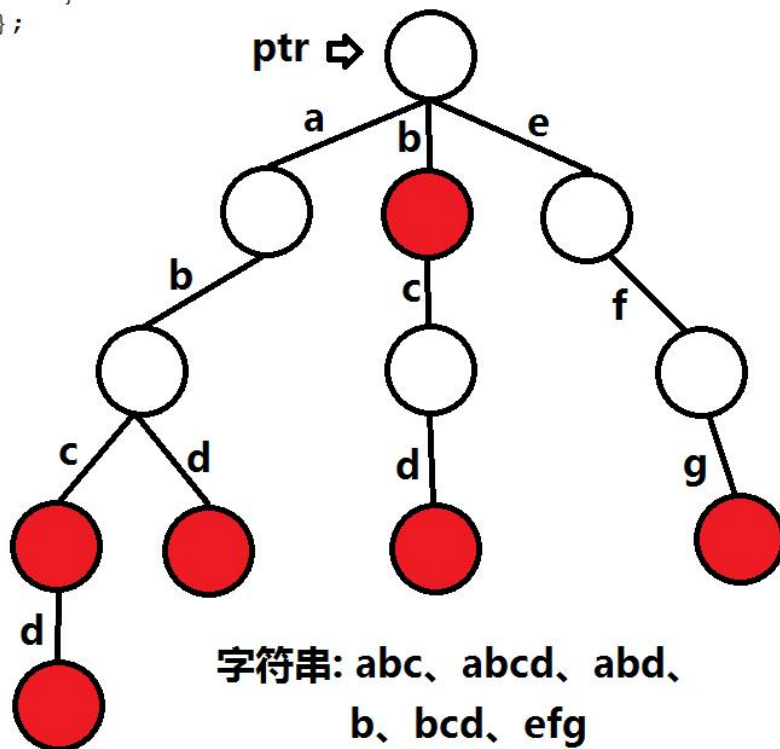
        while(*word){
            int pos = *word - 'a';

            if (!ptr->child[pos]){
                return false;
            }
            ptr = ptr->child[pos];
            word++;
        }
        return ptr->is_end;
    }
private:
    TrieNode *new_node(){
        TrieNode *node = new TrieNode();
        _node_vec.push_back(node);
        return node;
    }
    std::vector<TrieNode *> _node_vec;
    TrieNode _root;
};
```

```
#include <string>
#include <vector>

#define TRIE_MAX_CHAR_NUM 26

struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```



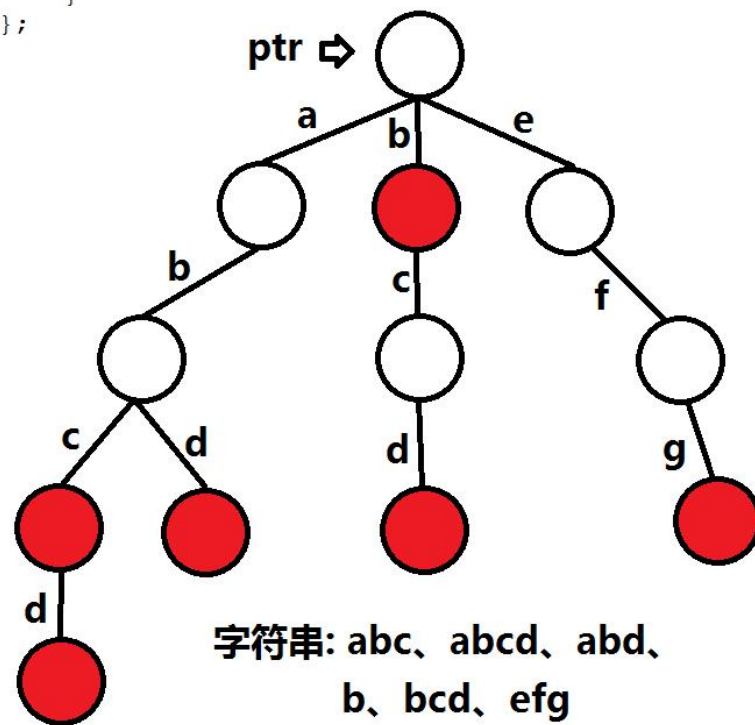
Trie树的前缀查询:课堂练习

```
class TrieTree{
public:
    TrieTree(){
    }
    ~TrieTree(){
        for (int i = 0; i < _node_vec.size(); i++){
            delete _node_vec[i];
        }
    }
    bool startsWith(const char *prefix){
        TrieNode *ptr = &_root;
        while( 1 ){
            int pos = 2
            if (!ptr->child[pos]){
                3
            }
            ptr = ptr->child[pos];
            4
        }
        5
    }
private:
    TrieNode *new_node(){
        TrieNode *node = new TrieNode();
        _node_vec.push_back(node);
        return node;
    }
    std::vector<TrieNode *> _node_vec;
    TrieNode _root;
};
```

```
#include <string>
#include <vector>

#define TRIE_MAX_CHAR_NUM 26

struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```



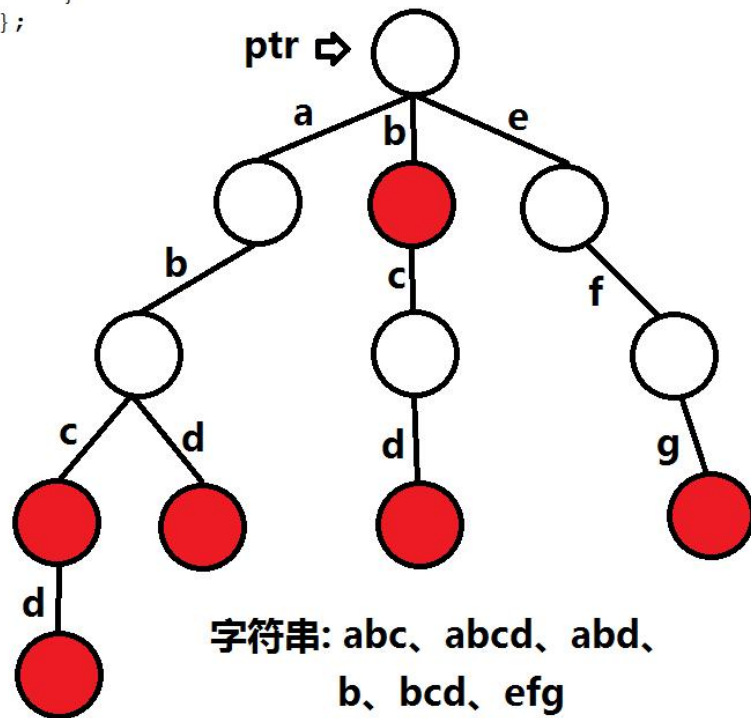
Trie树的前缀查询:实现

```
class TrieTree{
public:
    TrieTree(){
    }
    ~TrieTree(){
        for (int i = 0; i < _node_vec.size(); i++){
            delete _node_vec[i];
        }
    }
    bool startsWith(const char *prefix){
        TrieNode *ptr = &_root;
        while ( *prefix ){
            int pos = *prefix - 'a';
            if (!ptr->child[pos]){
                return false;
            }
            ptr = ptr->child[pos];
            prefix++;
        }
        return true;
    }
private:
    TrieNode *new_node(){
        TrieNode *node = new TrieNode();
        _node_vec.push_back(node);
        return node;
    }
    std::vector<TrieNode *> _node_vec;
    TrieNode _root;
};
```

```
#include <string>
#include <vector>

#define TRIE_MAX_CHAR_NUM 26

struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false){
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++){
            child[i] = 0;
        }
    }
};
```



Trie树的整体测试

```
int main(){
    TrieTree trie_tree;
    trie_tree.insert("abcd");
    trie_tree.insert("abc");
    trie_tree.insert("abd");
    trie_tree.insert("b");
    trie_tree.insert("bcd");
    trie_tree.insert("efg");
    printf("preorder_trie:\n");
    preorder_trie(trie_tree.root(), 0);
    printf("\n");
    std::vector<std::string> word_list;
    std::string word;
    printf("All words:\n");
    get_all_word_from_trie(trie_tree.root(), word, word_list);
    for (int i = 0; i < word_list.size(); i++){
        printf("%s\n", word_list[i].c_str());
    }
    printf("\n");
    printf("Search:\n");
    printf("abc : %d\n", trie_tree.search("abc"));
    printf("abcd : %d\n", trie_tree.search("abcd"));
    printf("bc : %d\n", trie_tree.search("bc"));
    printf("b : %d\n", trie_tree.search("b"));
    printf("\n");
    printf("ab : %d\n", trie_tree.startsWith("ab"));
    printf("abc : %d\n", trie_tree.startsWith("abc"));
    printf("bc : %d\n", trie_tree.startsWith("bc"));
    printf("fg : %d\n", trie_tree.startsWith("fg"));
    return 0;
}
```

```
preorder_trie:
a
---b
-----c(end)
-----d(end)
-----d(end)
b(end)
---c
-----d(end)
e
---f
-----g(end)
```

```
All words:
abc
abcd
abd
b
bcd
efg
```

```
Search:
abc : 1
abcd : 1
bc : 0
b : 1

ab : 1
abc : 1
bc : 1
fg : 0
```

例1: 实现trie树

实现一个**trie树**，包括insert(插入单词)、search(搜索单词)、startsWith(检查是否包含前缀)。输入的字符串只包含**小写字符(a-z)**。

```
class Trie {
public:
    Trie() {
    }
    void insert(std::string word) {
    }
    bool search(std::string word) {
    }
    bool startsWith(std::string prefix) {
    }
};
```

选自 **LeetCode 208. Implement Trie (Prefix Tree)**

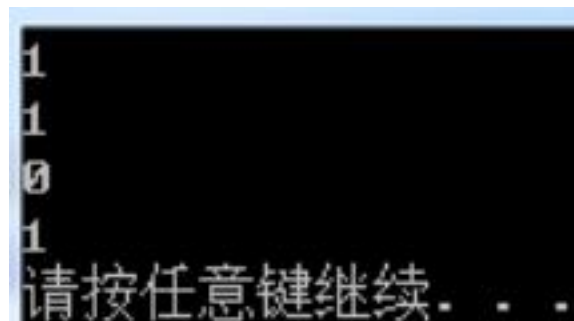
<https://leetcode.com/problems/implement-trie-prefix-tree/description/>

难度:**Medium**

例1: 实现、测试与leetcode提交结果

```
class Trie {
public:
    Trie() {
    }
    void insert(std::string word) {
        _trie_tree.insert(word.c_str());
    }
    bool search(std::string word) {
        return _trie_tree.search(word.c_str());
    }
    bool startsWith(std::string prefix) {
        return _trie_tree.startsWith(prefix.c_str());
    }
private:
    TrieTree _trie_tree;
};

int main() {
    Trie trie;
    trie.insert("abcde");
    printf("%d\n", trie.search("abcde"));
    printf("%d\n", trie.startsWith("abc"));
    printf("%d\n", trie.startsWith("abcdef"));
    printf("%d\n", trie.startsWith("abcde"));
    return 0;
}
```



Implement Trie (Prefix Tree)

Submission Details

14 / 14 test cases passed.

Status: **Accepted**

Runtime: 59 ms

Submitted: 0 minutes ago

例2:添加与查找单词

设计一个**数据结构**，支持如下两个操作：

1)添加单词: `addWord(word)`

2)搜索单词: `bool search(word)`

搜索单词时，可以按照**普通的方式**搜索单词(原始单词)，或**正则表达式的方式**

搜索单词，添加的单词**只包含小写字母**'a'-'z'；搜索时，只包含小写字母'a'-'z'或'.'。

'.'代表**任意一个小写字母**。

例如：

```
addWord("bad")
addWord("dad")
addWord("mad")
search("pad") -> false
search("bad") -> true
search(".ad") -> true
search("b..") -> true
```

```
class WordDictionary {
public:
    WordDictionary() {
    }
    void addWord(std::string word) {
    }
    bool search(std::string word) {
    }
};
```

选自 **LeetCode 211. Add and Search Word - Data structure design**

<https://leetcode.com/problems/add-and-search-word-data-structure-design/description/>

难度:**Medium**

例2:思考

```
class TrieTree{
public:
    TrieTree() {
    }
    ~TrieTree() {
        for (int i = 0; i < _node_vec.size(); i++) {
            delete _node_vec[i];
        }
    }
    void insert(const char *word) {
        TrieNode *ptr = &_root;
        while(*word) {
            int pos = *word - 'a';
            if (!ptr->child[pos]) {
                ptr->child[pos] = new_node();
            }
            ptr = ptr->child[pos];
            word++;
        }
        ptr->is_end = true;
    }
    TrieNode *root() {
        return &_root;
    }
private:
    TrieNode *new_node() {
        TrieNode *node = new TrieNode();
        _node_vec.push_back(node);
        return node;
    }
    std::vector<TrieNode *> _node_vec;
    TrieNode _root;
};
```

```
#include <vector>
#define TRIE_MAX_CHAR_NUM 26

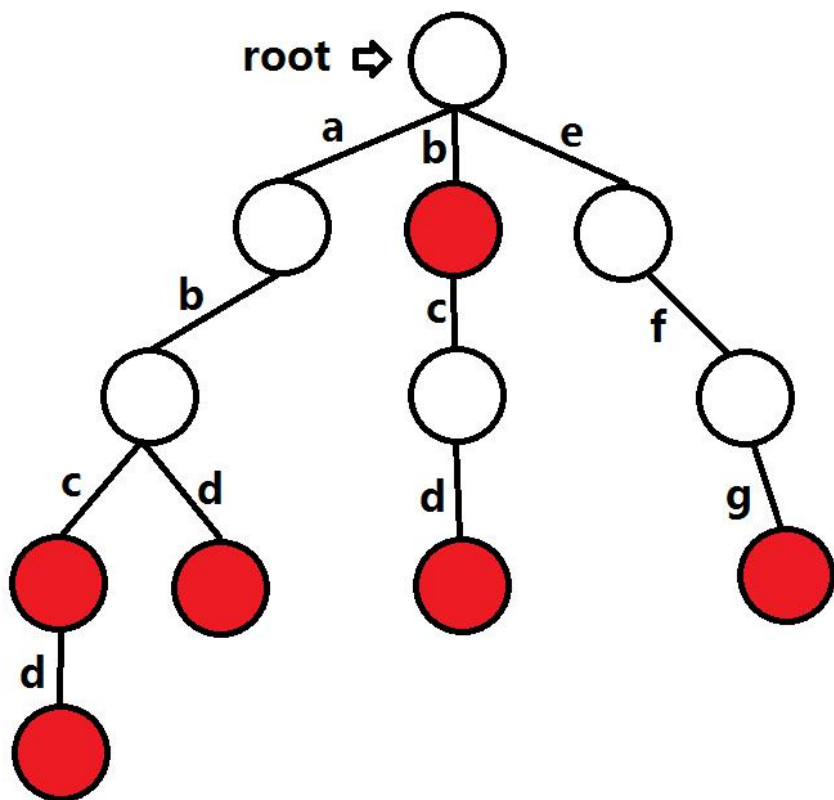
struct TrieNode{
    TrieNode *child[TRIE_MAX_CHAR_NUM];
    bool is_end;
    TrieNode() : is_end(false) {
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++) {
            child[i] = 0;
        }
    }
};

#include <string>

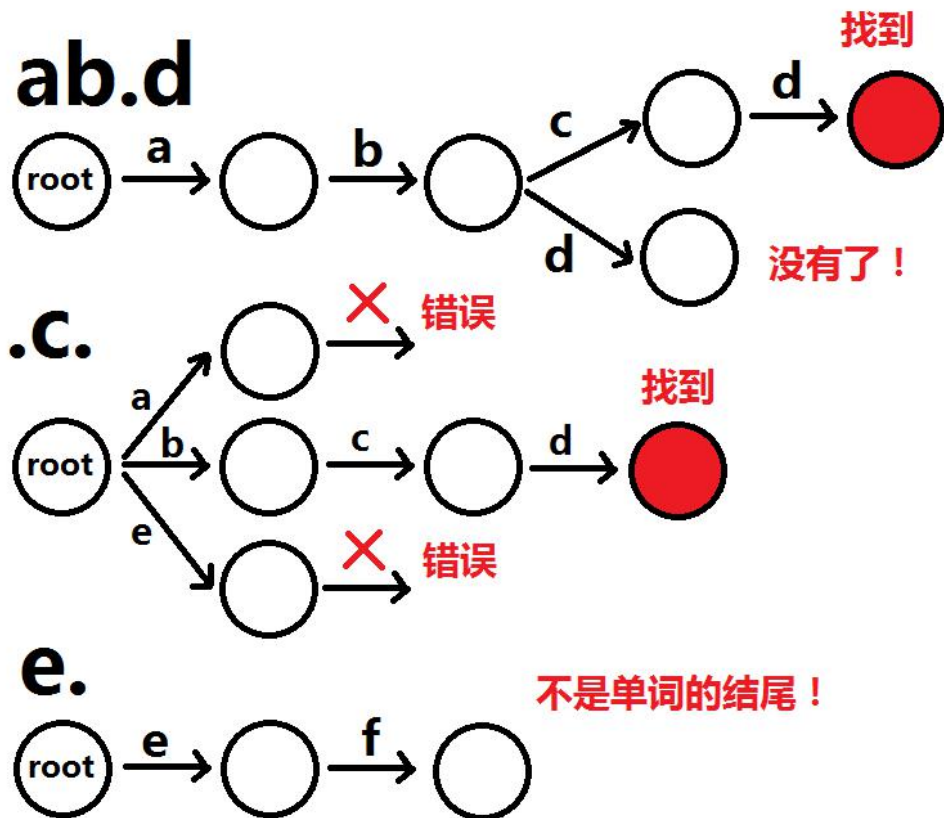
class WordDictionary {
public:
    WordDictionary() {
    }
    void addWord(std::string word) {
        _trie_tree.insert(word.c_str());
    }
    bool search(std::string word) {
        //使用trie树对字符串进行存储与查找，
        //对于存储(添加单词)，即为规范的trie添加单词的方法；
        //思考当出现'.'时，如何进行搜索？
    }
private:
    TrieTree _trie_tree;
};
```

例2:分析:深度搜索trie树

字符串: abc、abcd、abd、b、bcd、efg



深度优先搜索trie树:



例2:算法思路

node为正在搜索的节点，word指向正在搜索的单词起始，查找成功返回真；否则返回假：

bool search_trie(TrieNode *node, const char *word):

当遍历到单词**结束时**(word指向'\0'):

如果node指向的节点标记为单词的结尾(is_end为真)

返回真

否则返回假

如果word指向'.':

遍历node的全部孩子指针:

如果孩子指针为真，继续递归深搜该孩子子树，单词指针向前移动一个位置，

如果递归深搜结果为真，则返回真。

否则('a'-'z'):

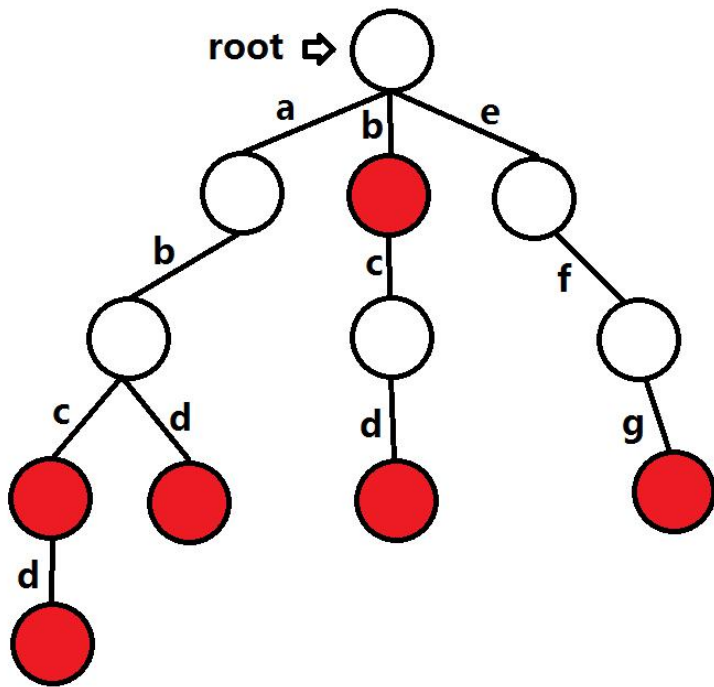
计算孩子位置 $pos = \text{当前字符} - 'a'$;

如果pos指向的孩子指针为真，继续递归深搜该孩子子树，单词指针向前移动一个位置，

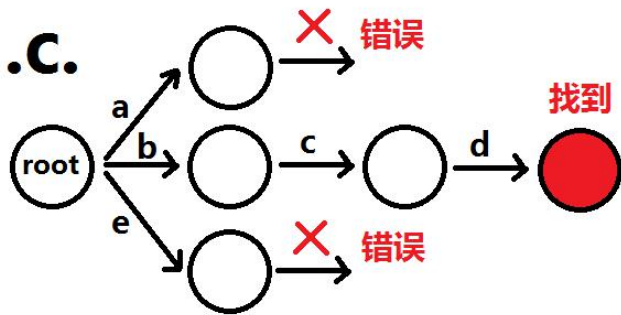
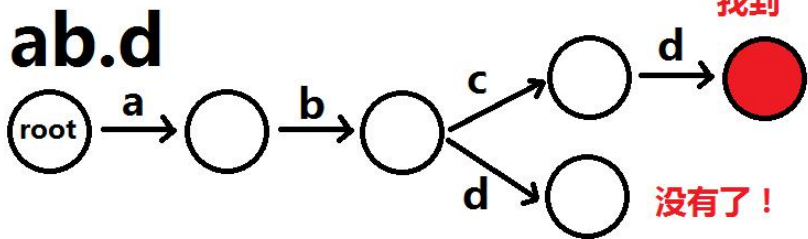
如果递归深搜结果为真，则返回真。

最终返回假

字符串: abc、abcd、abd、b、bcd、efg



深度优先搜索trie树:



```
bool search_trie(TrieNode *node, const char *word) {
```

```
    if (  ) {
```

```
        if (  ) {
```

```
            return true;
```

```
        }
```

```
        
```

```
    }
```

```
    if (*word == '.') {
```

```
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++) {
```

```
            if (node->child[i] &&
```

```
                search_trie( ,  )) {
```

```
                    return true;
```

```
            }
```

```
        }
```

```
    }
```

```
    else {
```

```
        int pos = 
```

```
        if (node->child[pos] &&
```

```
            search_trie( ,  )) {
```

```
                return true;
```

```
            }
```

```
    }
```

```
    return false;
```

```
}
```

例2:课堂练习

3分钟时间填写代码，
有问题随时提出！

例2:实现

```
bool search_trie(TrieNode *node, const char *word) {
    if (*word == '\0') {
        if (node->is_end) {
            return true;
        }
        return false;
    }
    if (*word == '.') {
        for (int i = 0; i < TRIE_MAX_CHAR_NUM; i++) {
            if (node->child[i] &&
                search_trie(node->child[i], word + 1)) {
                return true;
            }
        }
    }
    else {
        int pos = *word - 'a';
        if (node->child[pos] &&
            search_trie(node->child[pos], word + 1)) {
            return true;
        }
    }
    return false;
}
```


例2:测试与leetcode提交结果

```
class WordDictionary {
public:
    WordDictionary() {
    }
    void addWord(std::string word) {
        _trie_tree.insert(word.c_str());
    }
    bool search(std::string word) {
        return search_trie(_trie_tree.root(), word.c_str());
    }
private:
    TrieTree _trie_tree;
};

int main() {
    WordDictionary word_dictionary;
    word_dictionary.addWord("bad");
    word_dictionary.addWord("dad");
    word_dictionary.addWord("mad");
    printf("%d\n", word_dictionary.search("pad"));
    printf("%d\n", word_dictionary.search("bad"));
    printf("%d\n", word_dictionary.search(".ad"));
    printf("%d\n", word_dictionary.search("b.."));
    return 0;
}
```

Add and Search Word - Data structure design

Submission Details

13 / 13 test cases passed.

Status: **Accepted**

Runtime: 79 ms

Submitted: 0 minutes ago

```
0
1
1
1
1
请按任意键继续. . .
```

课间休息10分钟

有问题提出！

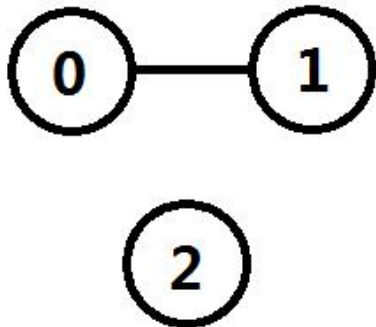
例3:朋友圈

有N个同学，他们之间**有些是朋友，有些不是**。"友谊"是可以**传递**的，例如A与B是朋友，B与C是朋友，那么A与C也是朋友；**朋友圈**就是完成"友谊"**传递**后的一组朋友。给定N*N的矩阵代表**同学间**是否是朋友，如果 $M[i][j] = 1$ 代表第i个学生与第j个学生是朋友，否则不是。求**朋友圈的个数**。

Input:

[[1,1,0],
[1,1,0],
[0,0,1]]

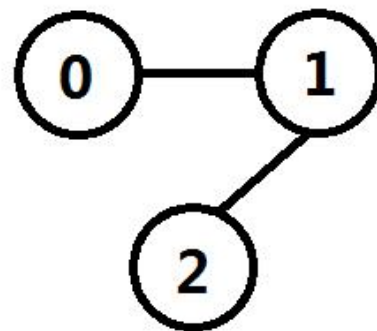
Output: 2



Input:

[[1,1,0],
[1,1,1],
[0,1,1]]

Output: 1



```
class Solution {  
public:  
    int findCircleNum(std::vector<std::vector<int> >& M) {  
    }  
};
```

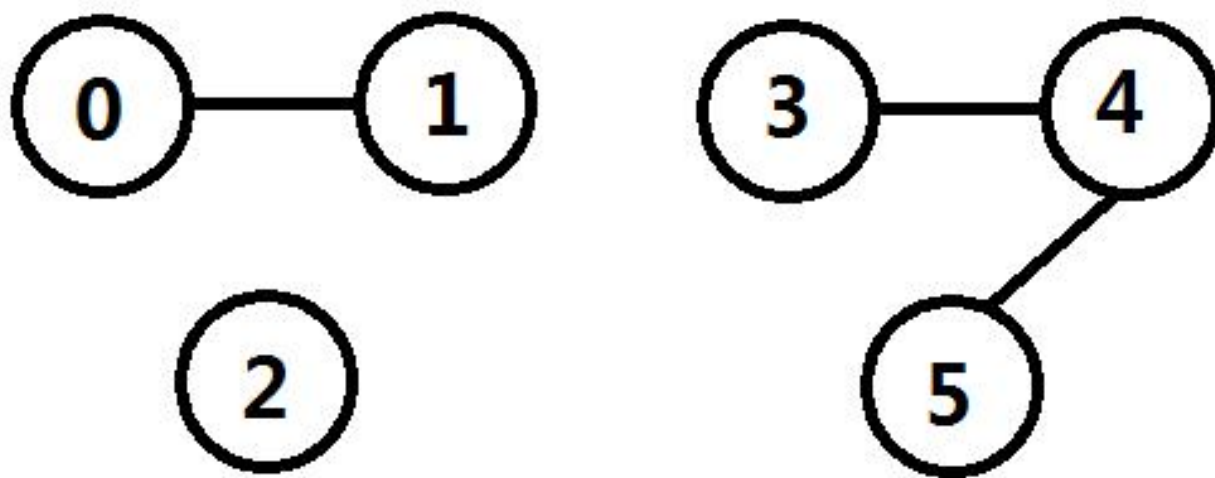
选自 **LeetCode 547. Friend Circles**

<https://leetcode.com/problems/friend-circles/description/>

难度: **Medium**

例3:方法1，图的深度优先搜索

该问题即求，图中有多少个**区域**是**相连**的，可以使用图的**深搜**或**广搜**的方法进行求解，思考对应算法：



回忆第8课搜索**例1-岛屿数量**与该题有什么样的**相似**之处？

例3:方法1， 课堂练习

```
#include <vector>

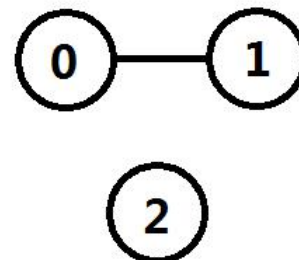
void DFS_graph(int u,
               std::vector<std::vector<int> >& graph,
               std::vector<int>& visit){
    1
    for (int i = 0; i < graph[u].size(); i++){
        if ( 2 && 3 ){
            DFS_graph(i, graph, visit);
        }
    }
}

class Solution {
public:
    int findCircleNum(std::vector<std::vector<int> >& M) {
        std::vector<int> visit(M.size(), 0);
        int count = 0;
        for (int i = 0; i < M.size(); i++){
            if ( 4 ){
                DFS_graph(i, M, visit);
                5
            }
        }
        return count;
    }
};
```

Input:

[[1,1,0],
[1,1,0],
[0,0,1]]

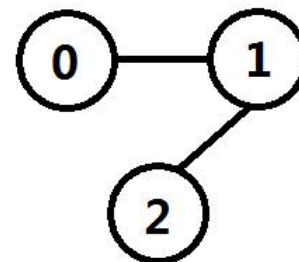
Output: 2



Input:

[[1,1,0],
[1,1,1],
[0,1,1]]

Output: 1



3分钟时间填写代码，
有问题随时提出！

例3:方法1，实现

```
#include <vector>

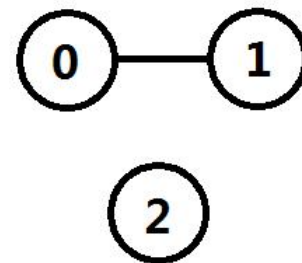
void DFS_graph(int u,
               std::vector<std::vector<int>> & graph,
               std::vector<int> & visit){
    visit[u] = 1;
    for (int i = 0; i < graph[u].size(); i++){
        if (visit[i] == 0 && graph[u][i] == 1){
            DFS_graph(i, graph, visit);
        }
    }
}

class Solution {
public:
    int findCircleNum(std::vector<std::vector<int>> & M) {
        std::vector<int> visit(M.size(), 0);
        int count = 0;
        for (int i = 0; i < M.size(); i++){
            if (visit[i] == 0){
                DFS_graph(i, M, visit);
                count++;
            }
        }
        return count;
    }
};
```

Input:

[[1,1,0],
[1,1,0],
[0,0,1]]

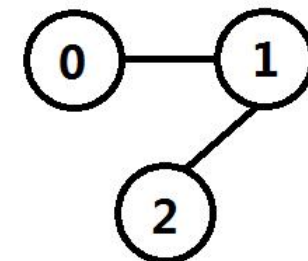
Output: 2



Input:

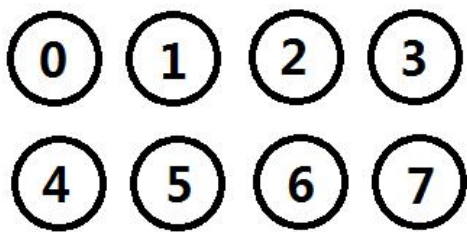
[[1,1,0],
[1,1,1],
[0,1,1]]

Output: 1



并查集概述

并查集(Union Find)，又称不相交集合(Disjiont Set)，它应用于N个元素的集合**求并**与**查询**问题，在该应用场景中，我们通常是在开始时让每个元素构成一个**单元素**的集合，然后按**一定顺序**将属于**同一组**的元素所在的集合**合并**，其间要**反复查找**一个元素在哪个集合中。虽然该问题并不复杂，但面对**极大的数据量**时，普通的数据结构往往无法解决，并查集就是解决该种问题**最为优秀的算法**。



Union(0, 5)

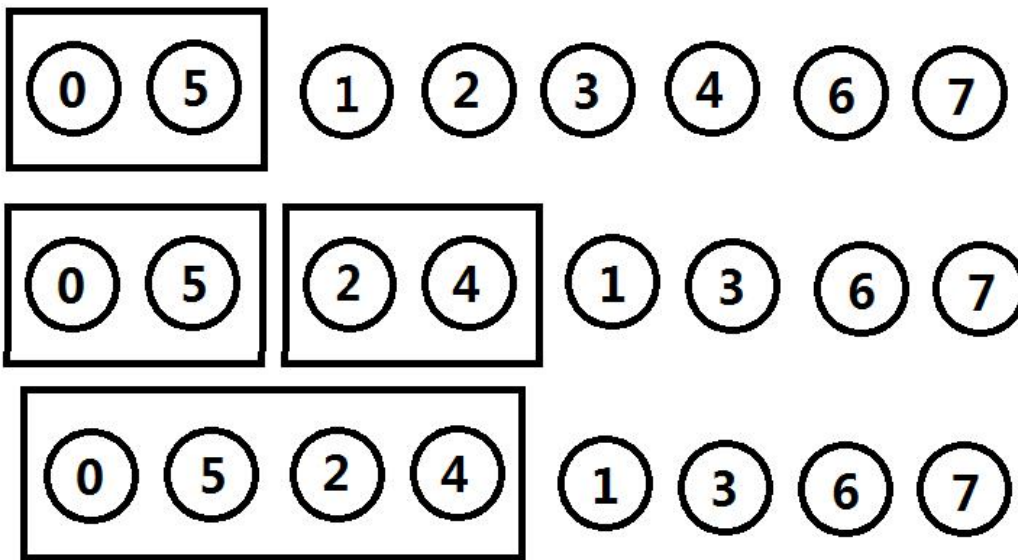
Find(0) ? Find(5) 相同集合

Find(2) ? Find(5) 不同集合

Union(2, 4)

Union(0, 4)

Find(2) ? Find(5) 相同集合



并查集功能分析

初始时:

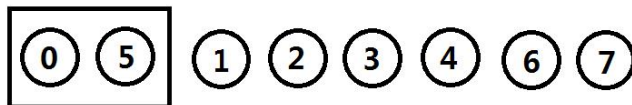
元素 0 1 2 3 4 5 6 7

集合 0 1 2 3 4 5 6 7

Union(0, 5)

元素 0 1 2 3 4 5 6 7

集合 0 1 2 3 4 0 6 7



或

元素 0 1 2 3 4 5 6 7

集合 5 1 2 3 4 5 6 7

Find(0) == 5

Find(5) == 5

Find(0) == Find(5) 在同一个集合 !

Find(2) == 2

Find(5) == 5

Find(2) != Find(5) 不在同一个集合!

Union(2, 4)

Union(0, 4)

元素 0 1 2 3 4 5 6 7

集合 0 1 0 3 0 0 6 7

或

元素 0 1 2 3 4 5 6 7

集合 5 1 5 3 5 5 6 7

或

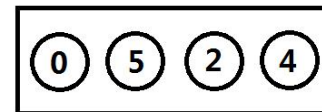
元素 0 1 2 3 4 5 6 7

集合 2 1 2 3 2 2 6 7

或

元素 0 1 2 3 4 5 6 7

集合 4 1 4 3 4 4 6 7



Find(2) == Find(5) 在同一个集合 !

数组实现并查集

```
class DisjointSet{
public:
    DisjointSet(int n){
        for (int i = 0; i < n; i++){
            _id.push_back(i);
        }
    }
    int find(int p){
        return _id[p]; //查询元素p属于哪个集合时直接返回id[i]
    }
    void union_(int p, int q){
        int pid = find(p);
        int qid = find(q); //合并时，若两个元素属于同一个集合
        if (pid == qid){ //则直接返回
            return;
        }
        for (int i = 0; i < _id.size(); i++){
            if (_id[i] == pid){
                _id[i] = qid;
            }
        } //将所有属于pid集合的全部改为qid对应的集合
    }
private:
    std::vector<int> _id;
};
```

复杂度O(n)

初始时:

元素 0 1 2 3 4 5 6 7

集合 0 1 2 3 4 5 6 7

Union(0, 5)

元素 0 1 2 3 4 5 6 7

集合 5 1 2 3 4 5 6 7

Find(0) == 5

Find(5) == 5

Find(0) == Find(5)

在同一个集合！

Find(2) == 2

Find(5) == 5

Find(2) != Find(5)

不在同一个集合！

数组实现并查集，测试

```
void print_set(){
    printf("元素: ");
    for (int i = 0; i < _id.size(); i++){
        printf("%d ", i);
    }
    printf("\n");
    printf("集合: ");
    for (int i = 0; i < _id.size(); i++){
        printf("%d ", _id[i]);
    }
    printf("\n");
}

private:
    std::vector<int> _id;
};

int main(){
    DisjointSet disjoint_set(8);
    disjoint_set.print_set();
    printf("Union(0, 5)\n");
    disjoint_set.union_(0, 5);
    disjoint_set.print_set();
    printf("Find(0) = %d, Find(5) = %d\n",
           disjoint_set.find(0), disjoint_set.find(5));
    printf("Find(2) = %d, Find(5) = %d\n",
           disjoint_set.find(2), disjoint_set.find(5));
    disjoint_set.union_(2, 4);
    disjoint_set.print_set();
    disjoint_set.union_(0, 4);
    disjoint_set.print_set();
    printf("Find(2) = %d, Find(5) = %d\n",
           disjoint_set.find(2), disjoint_set.find(5));
    return 0;
}
```

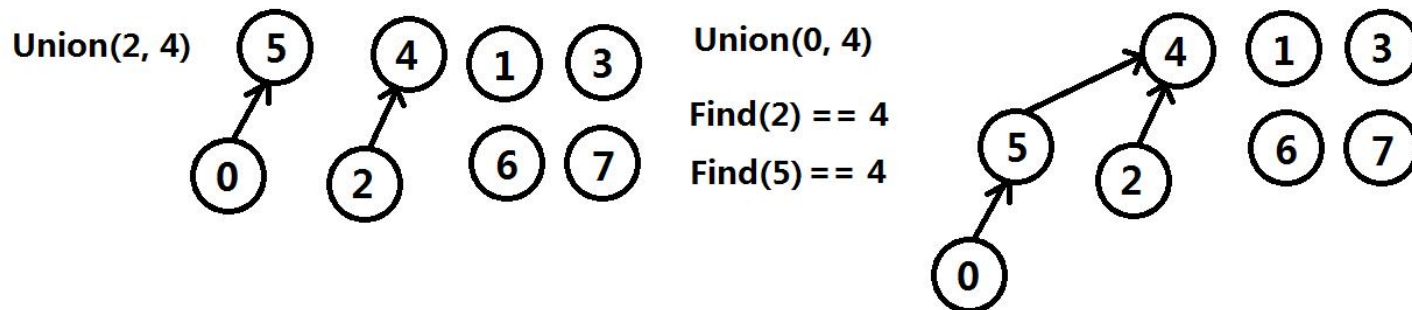
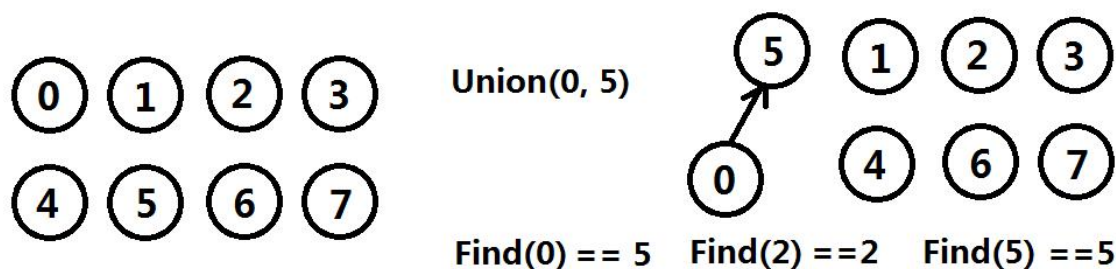
```
元素: 0 1 2 3 4 5 6 7
集合: 0 1 2 3 4 5 6 7
Union(0, 5)
元素: 0 1 2 3 4 5 6 7
集合: 5 1 2 3 4 5 6 7
Find(0) = 5, Find(5) = 5
Find(2) = 2, Find(5) = 5
元素: 0 1 2 3 4 5 6 7
集合: 5 1 4 3 4 5 6 7
元素: 0 1 2 3 4 5 6 7
集合: 4 1 4 3 4 4 6 7
Find(2) = 4, Find(5) = 4
请按任意键继续. . .
```


并查集，森林实现

使用**森林**存储集合之间的关系，属于**同一集合**的**不同元素**，都有一个**相同的根节点**，代表着这个集合。

当进行**查找**某元素属于哪个集合时，即遍历该元素到**根节点**，返回**根节点**所代表的集合；在遍历过程中使用**路径压缩**的优化算法，使整体树的形状**更加扁平**，从而优化**查询**的时间复杂度。

当进行**合并**时，即将两颗子树**合为**一颗树，将一颗子树的根节点**指向**另一颗子树的根节点；在合并时可按子树的**大小**，将规模较小的子树**合并**到规模较大的子树上，从而使树规模**更加平衡**，从而优化未来**查询**的时间复杂度。



并查集，查找算法

在查找时，普通的查找即通过id数组遍历至**根节点**：

当p 与当前 集合id[p] **不同时**(直到p与id[p]相同时跳出循环)，进行循环：

$p = id[p]$

返回p的值

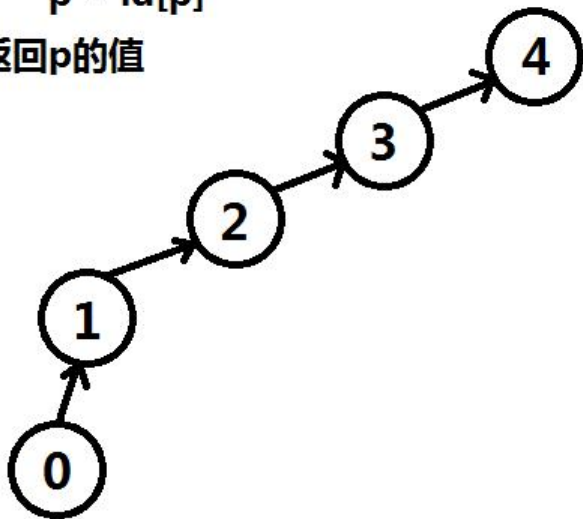
在查找时增加**路径压缩**的优化算法：

当p 与当前 集合id[p] **不同时**(直到p与id[p]相同时跳出循环)，进行循环：

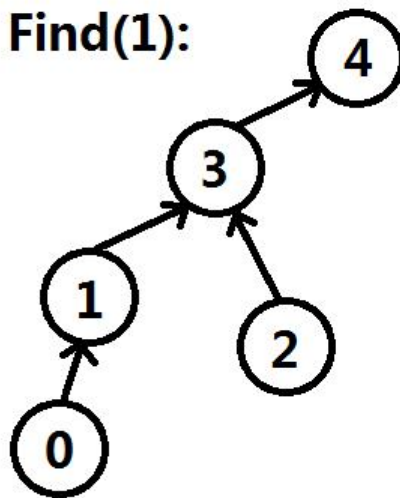
将p的父节点id[p]更新为id[p]的父节点id[id[p]]，

$p = id[p]$

返回p的值

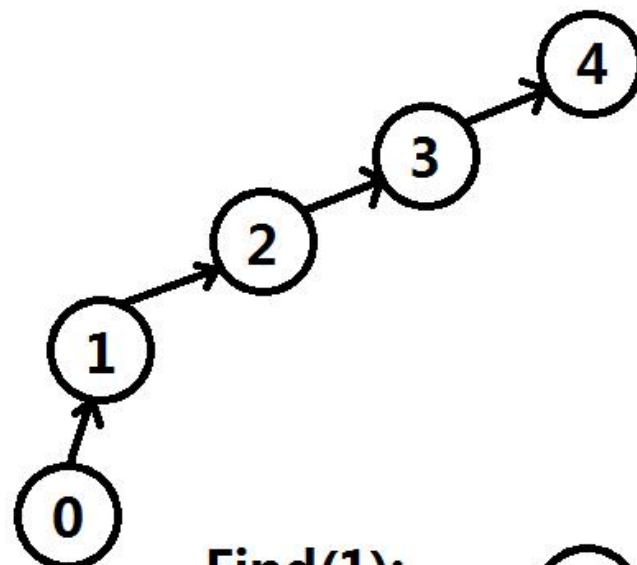


Find(1):

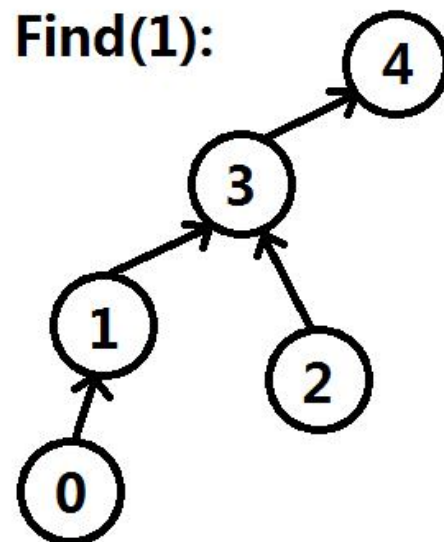


并查集，查找算法，课堂练习

```
class DisjointSet{
public:
    DisjointSet(int n){
        for (int i = 0; i < n; i++){
            _id.push_back(i);
            _size.push_back(1);
        }
        _count = n;
    }
    int find(int p){
        while (1){
            2
            p = _id[p];
            3
        }
    }
    void union_(int p, int q){
    }
private:
    std::vector<int> _id;
    std::vector<int> _size;
    int _count;
};
```

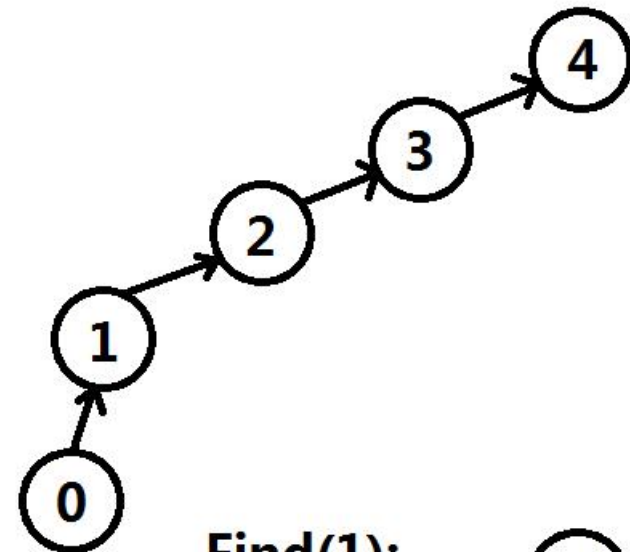


Find(1):

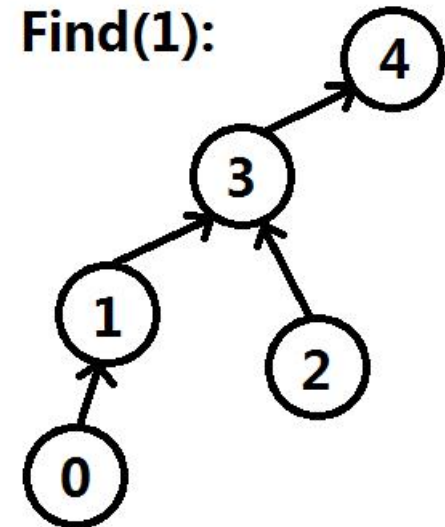


并查集，查找算法，实现

```
class DisjointSet{
public:
    DisjointSet(int n){
        for (int i = 0; i < n; i++){
            _id.push_back(i);
            _size.push_back(1);
        }
        _count = n;
    }
    int find(int p){
        while (p != _id[p]){
            _id[p] = _id[_id[p]];
            p = _id[p];
        }
        return p;
    }
    void union_(int p, int q){
    }
private:
    std::vector<int> _id;
    std::vector<int> _size;
    int _count;
};
```



Find(1):



并查集，合并算法

当进行集合的**合并**时，即将两颗子树**合为**一颗树，将一颗子树的根节点**指向**另一颗子树的**根节点**；

在合并时可按**子树的大小**，将规模较小的子树**合并到**规模较大的子树上，从而使树**更加平衡**，从而**优化未来查询**的时间复杂度。

合并p所在的集合与q所在的集合：

查找p所在集合的根， $i = \text{find}(p)$ ；

查找q所在集合的根， $j = \text{find}(q)$ ；

如果 i 与 j 相同，则**直接返回**；

如果 i 所在子树规模**小于** j 所在子树规模：

将 i 的根**指向** j ；

j 的规模**增加** i 子树的规模

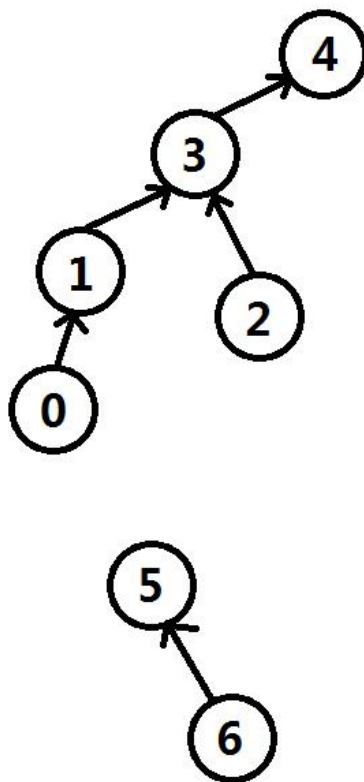
否则：

将 j 的根**指向** i ；

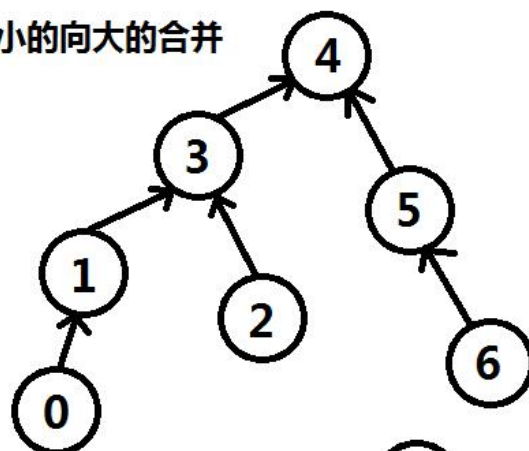
i 的规模**增加** j 子树的规模

子树个数减1

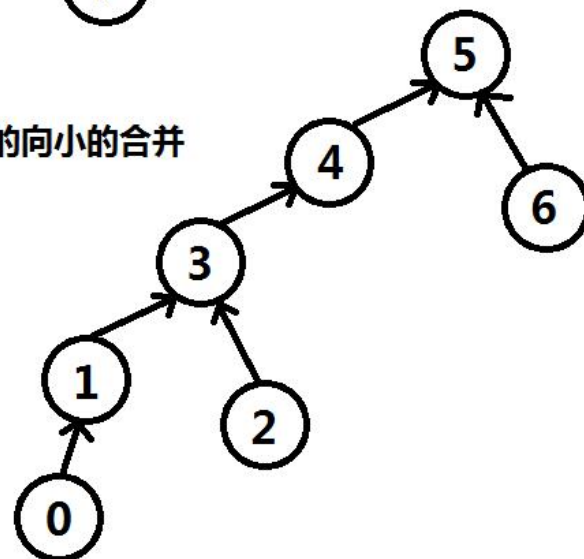
Union(1, 6)



小的向大的合并



大的向小的合并



并查集，合并算法，课堂练习

```
#include <vector>
```

```
class DisjointSet{  
public:
```

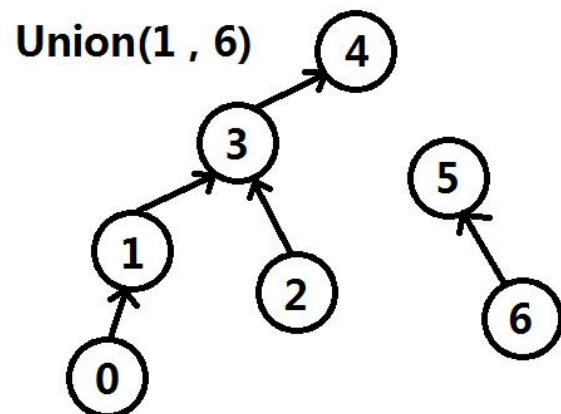
```
    DisjointSet(int n){  
        for (int i = 0; i < n; i++){  
            _id.push_back(i);  
            _size.push_back(1);  
        }  
        _count = n;
```

```
    }  
    int find(int p){  
        while(p != _id[p]){  
            _id[p] = _id[_id[p]];  
            p = _id[p];  
        }  
        return p;  
    }  
}
```

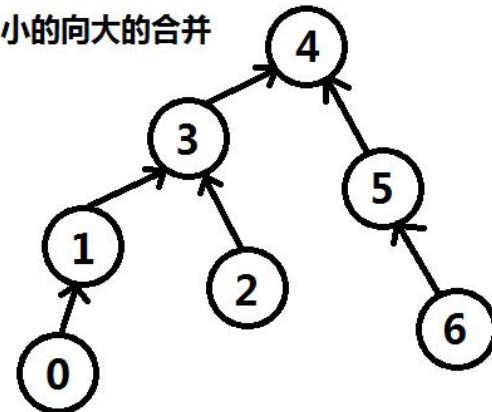
```
private:
```

```
    std::vector<int> _id;  
    std::vector<int> _size;  
    int _count;  
};
```

```
void union_(int p, int q){  
    int i = find(p);  
    int j = find(q);  
    if (1){  
        return;  
    }  
    if (_size[i] < _size[j]){  
        2  
        3  
    }  
    else{  
        4  
        5  
    }  
    _count--;  
}
```



小的向大的合并



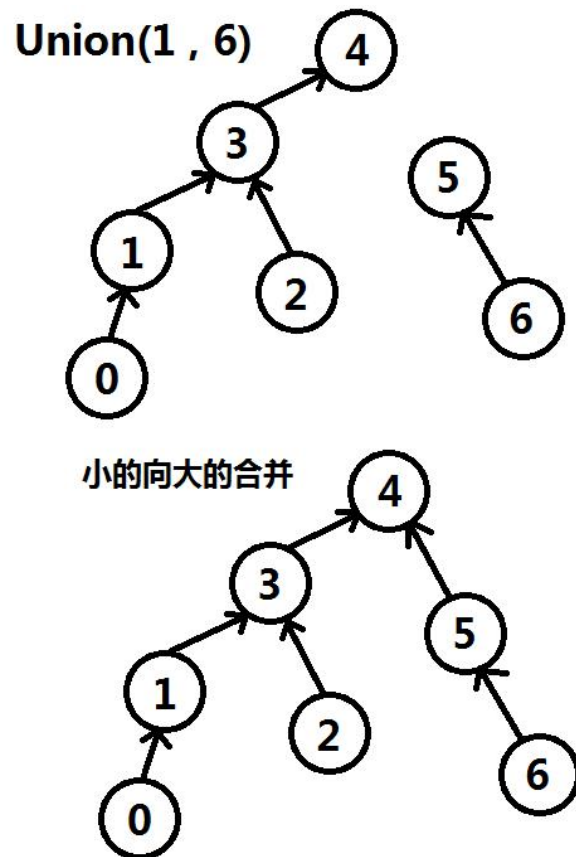
并查集，合并算法，实现

```
#include <vector>

class DisjointSet{
public:
    DisjointSet(int n){
        for (int i = 0; i < n; i++){
            _id.push_back(i);
            _size.push_back(1);
        }
        _count = n;
    }
    int find(int p){
        while(p != _id[p]){
            _id[p] = _id[_id[p]];
            p = _id[p];
        }
        return p;
    }

private:
    std::vector<int> _id;
    std::vector<int> _size;
    int _count;
};
```

```
void union_(int p, int q){
    int i = find(p);
    int j = find(q);
    if (i == j){
        return;
    }
    if (_size[i] < _size[j]){
        _id[i] = j;
        _size[j] += _size[i];
    }
    else{
        _id[j] = i;
        _size[i] += _size[j];
    }
    _count--;
```



并查集，整体测试

```
int main() {
    DisjointSet disjoint_set(8);
    disjoint_set.print_set();
    printf("Union(0, 5)\n");
    disjoint_set.union_(0, 5);
    disjoint_set.print_set();
    printf("Find(0) = %d, Find(5) = %d\n",
           disjoint_set.find(0), disjoint_set.find(5));
    printf("Find(2) = %d, Find(5) = %d\n",
           disjoint_set.find(2), disjoint_set.find(5));
    disjoint_set.union_(2, 4);
    disjoint_set.print_set();
    disjoint_set.union_(0, 4);
    disjoint_set.print_set();
    printf("Find(2) = %d, Find(5) = %d\n",
           disjoint_set.find(2), disjoint_set.find(5));
    return 0;
}
```

```
元素: 0 1 2 3 4 5 6 7
集合: 0 1 2 3 4 5 6 7
Union(0, 5)
元素: 0 1 2 3 4 5 6 7
集合: 0 1 2 3 4 0 6 7
Find(0) = 0, Find(5) = 0
Find(2) = 2, Find(5) = 0
元素: 0 1 2 3 4 5 6 7
集合: 0 1 2 3 2 0 6 7
元素: 0 1 2 3 4 5 6 7
集合: 0 1 0 3 2 0 6 7
Find(2) = 0, Find(5) = 0
请按任意键继续. . .
```

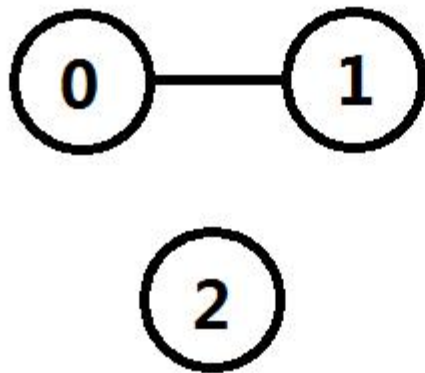
例3:方法2，并查集

传入代表朋友**友谊**的二维数组M;
设置并查集，**并查集**大小为M.size();
对任意两个学生i,j，如果他们间**存在友谊**($M[i][j] == 1$)，则将它们进行**合并**;
返回并查集**集合个数**。

Input:

[[1,1,0],
[1,1,0],
[0,0,1]]

Output: 2

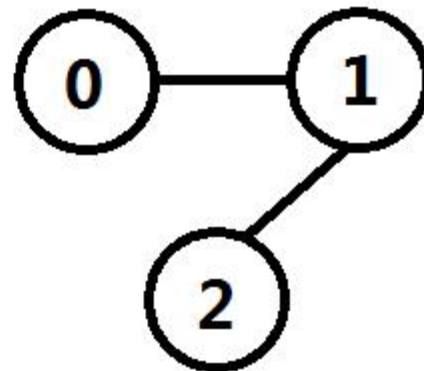


Union(0, 1)

Input:

[[1,1,0],
[1,1,1],
[0,1,1]]

Output: 1



Union(0, 1)

Union(1, 2)

例3:方法2，并查集，整体实现

```
class Solution {
public:
    int findCircleNum(std::vector<std::vector<int> >& M) {
        DisjointSet disjoint_set(M.size());
        for (int i = 0; i < M.size(); i++) {
            for (int j = i + 1; j < M.size(); j++) {
                if (M[i][j]) {
                    disjoint_set.union_(i, j);
                }
            }
        }
        return disjoint_set.count();
    }
};
```


例3:测试与leetcode提交结果

```
int main() {
    int test[][3] = {{1, 1, 0}, {1, 1, 0}, {0, 0, 1}};
    std::vector<std::vector<int> > M(3, std::vector<int>(3, 0));
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            M[i][j] = test[i][j];
        }
    }
    Solution solve;
    printf("%d\n", solve.findCircleNum(M));
    return 0;
}
```

Friend Circles

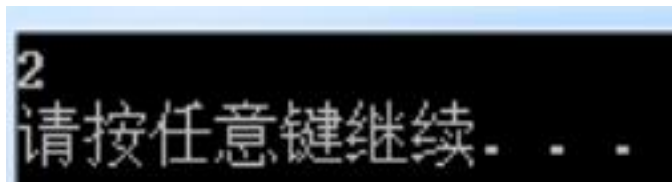
Submission Details

113 / 113 test cases passed.

Status: **Accepted**

Runtime: 19 ms

Submitted: 0 minutes ago



例4:区域和的查询

给定一个**整数数组**nums，求这个整数数组中，**下标i到下标j**之间的数字和($i \leq j$)， $a[i] + a[i+1] + \dots + a[j]$ 。在求和的过程中，可能需要**更新数组**的某个元素 $a[i]$ 。

```
class NumArray {  
public:  
    NumArray(std::vector<int> nums) {  
    }  
    void update(int i, int val) {  
    }  
    int sumRange(int i, int j) {  
    }  
};
```

数组 nums [1, 3, 5]
sumRange(0, 2) -> 9
update(1, 2)
sumRange(0, 2) -> 8

选自 **LeetCode 307. Range Sum Query – Mutable**

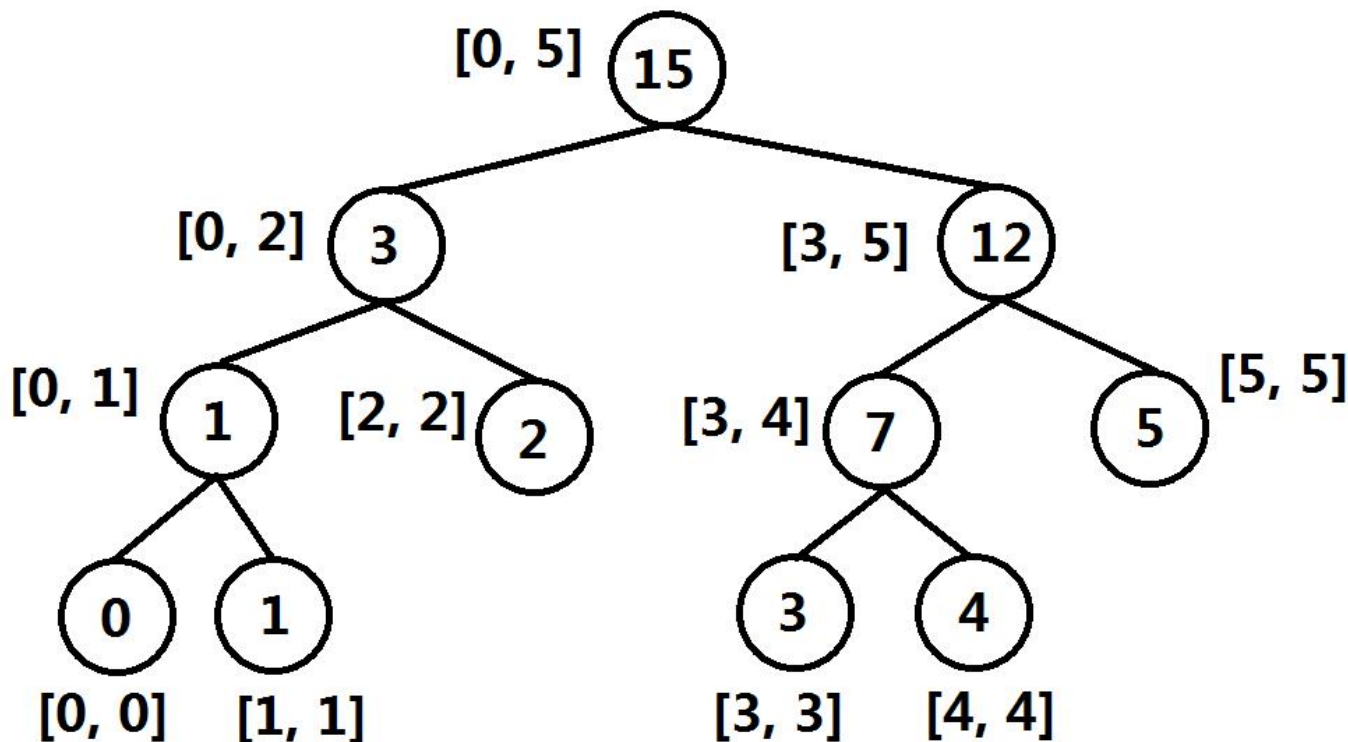
<https://leetcode.com/problems/range-sum-query-mutable/description/>

难度:**Medium**

例4: 线段树

线段树是一种**平衡二叉搜索树(完全二叉树)**，它将一个**线段区间**划分成一些**单元区间**。对于线段树中的每一个**非叶子节点** $[a,b]$ ，它的**左儿子**表示的区间为 $[a,(a+b)/2]$ ，**右儿子**表示的区间为 $[(a+b)/2+1,b]$ ，最后的**叶子节点**数目为 N ，与数组下标对应。线段树的一般包括**建立、查询、插入、更新**等操作，建立规模为 N 的时间复杂度是 $O(N\log N)$ ，其他操作时间复杂度为 $O(\log N)$ 。

将数组 $[0, 1, 2, 3, 4, 5]$ 保存在存储**区间和**的线段树中



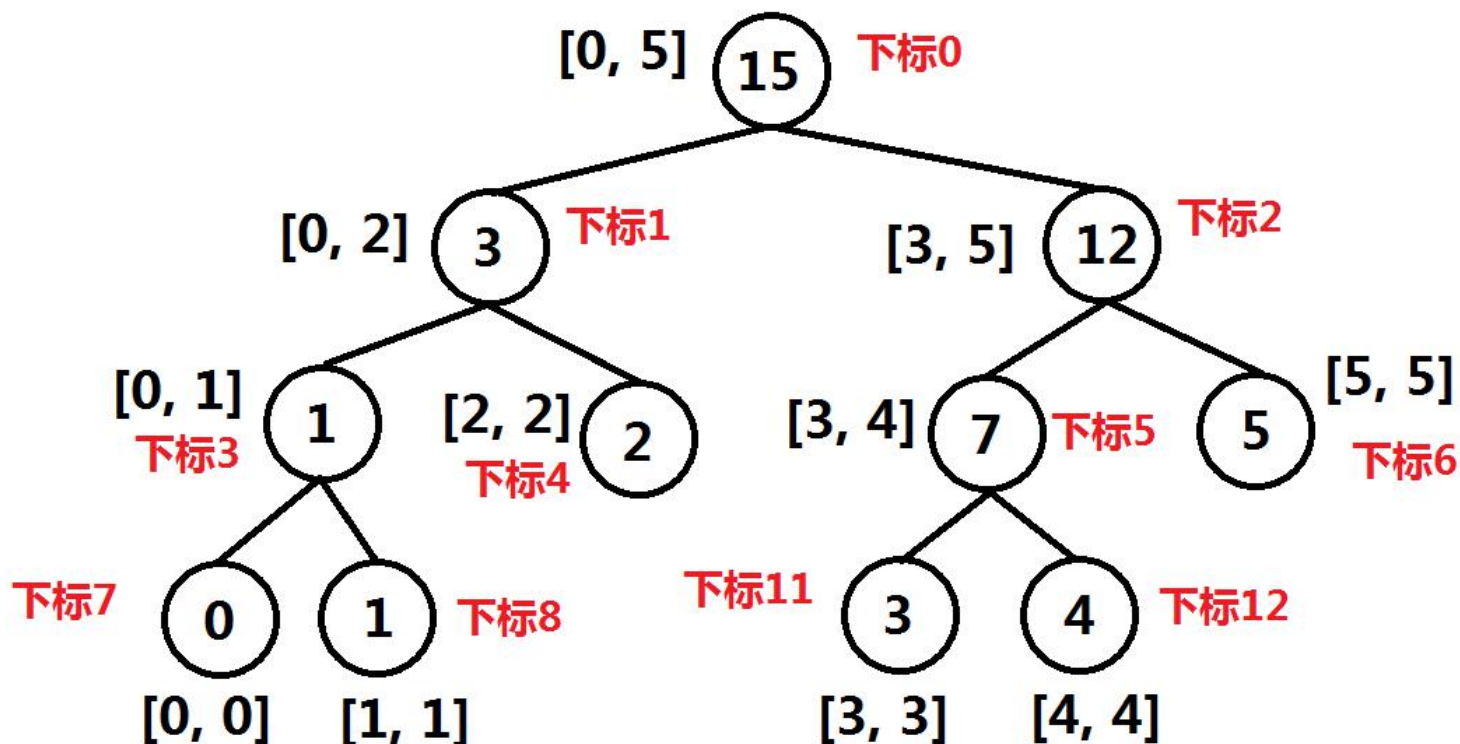
例4: 线段树的保存

由于线段树是**完全二叉树**，线段树可以使用**数组**保存，如：

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

[15, 3, 12, 1, 2, 7, 5, 0, 1, 0, 0, 3, 4]

将数组[0, 1, 2, 3, 4, 5]保存在存储**区间和**的线段树中



它的根节点下标为0，设某个节点的下标为 i ，它的左孩子下标为 $2*i+1$ ，右孩子下标为 $2*i+2$

例4: 线段树的构造

将数组[0, 1, 2, 3, 4, 5]保存在存储区间和的线段树中

传入参数:

线段树数组value(存储区间sum);

原始数组nums;

当前线段(节点)在线段树数组(value)中的下标pos;

当前线段的左端点left、右端点right

如果左端点left与右端点right相同, 说明到达叶结点:

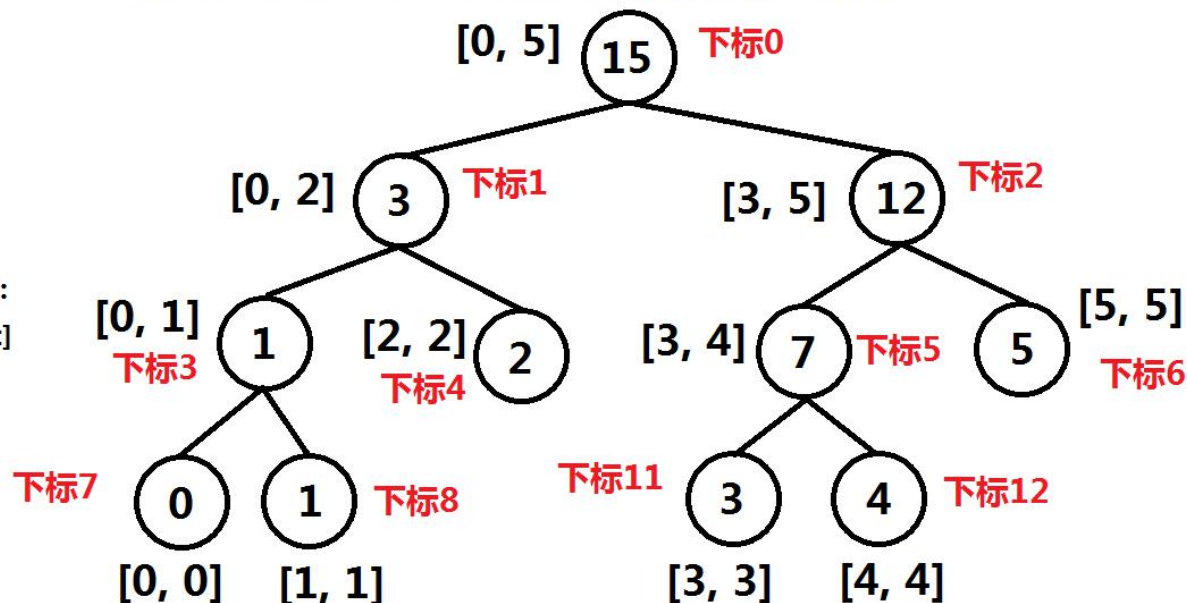
赋值value[pos]为nums[left]或者nums[right]

计算线段中心: $\text{mid} = (\text{left} + \text{right}) / 2$

递归建立左子树线段 [left, mid]

递归建立右子树线段 [mid + 1, right]

计算value[pos], 为左右子树代表的区间值的和



它的根节点下标为0, 设某个节点的下标为i, 它的左孩子下标为 $2*i+1$, 右孩子下标为 $2*i+2$

例4: 线段树的构造, 课堂练习

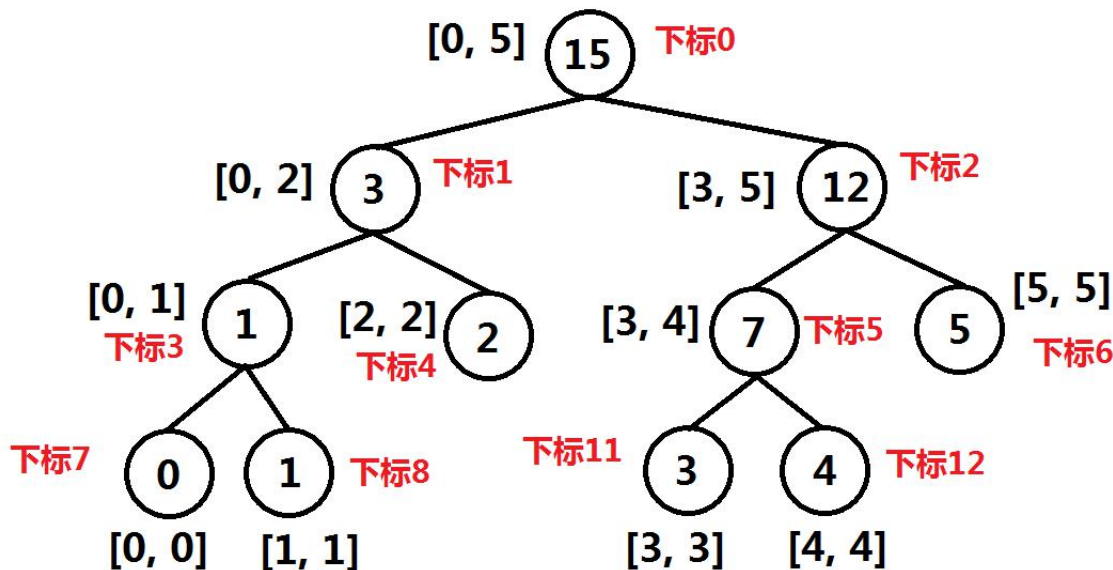
```
void build_segment_tree(std::vector<int> &value,  
                        std::vector<int> &nums,  
                        int pos, int left, int right){  
    if ( 1 ){  
        value[pos] = nums[left];  
        return;  
    }  
    int mid = 2;  
    build_segment_tree(value, nums, 3, left, mid);  
    build_segment_tree(value, nums, 4, mid + 1, right);  
    value[pos] = 5;  
}
```

例4: 线段树的构造, 实现

```
void build_segment_tree(std::vector<int> &value,  
                        std::vector<int> &nums,  
                        int pos, int left, int right){  
    if (left == right){  
        value[pos] = nums[left];  
        return;  
    }  
    int mid = (left + right) / 2;  
    build_segment_tree(value, nums, pos * 2 + 1, left, mid);  
    build_segment_tree(value, nums, pos * 2 + 2, mid + 1, right);  
    value[pos] = value[pos * 2 + 1] + value[pos * 2 + 2];  
}
```

例4: 线段树的遍历

```
void print_segment_tree(std::vector<int> &value,  
    int pos, int left, int right, int layer){  
    for (int i = 0; i < layer; i++){  
        printf("---");  
    }  
    printf("[%d %d][%d] : %d\n", left, right, pos, value[pos]);  
    if (left == right){  
        return;  
    }  
    int mid = (left + right) / 2;  
    print_segment_tree(value, pos * 2 + 1, left, mid, layer + 1);  
    print_segment_tree(value, pos * 2 + 2, mid + 1, right, layer + 1);  
}
```



```
segment_tree:  
[0 5][0] : 15  
---[0 2][1] : 3  
-----[0 1][3] : 1  
-----[0 0][7] : 0  
-----[1 1][8] : 1  
-----[2 2][4] : 2  
---[3 5][2] : 12  
-----[3 4][5] : 7  
-----[3 3][11] : 3  
-----[4 4][12] : 4  
-----[5 5][6] : 5
```

例4: 线段树的求和

传入参数:

线段树数组value(存储区间sum);

当前线段(节点)在线段树数组(value)中的下标pos;

当前线段的左端点left、右端点right

待求和区间的左端点qlleft、右端点qright

返回值:

该区间的和的值。

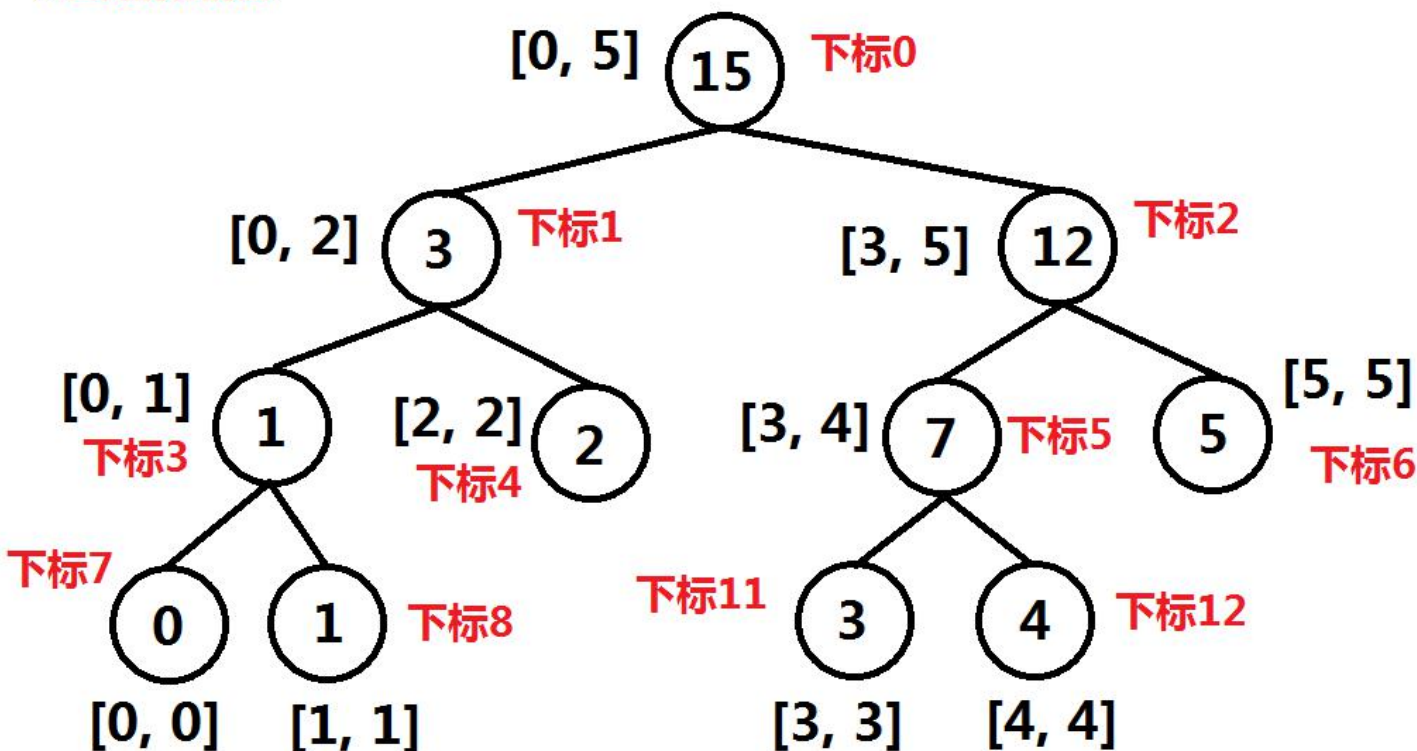
如果区间[left, right]与[qleft, qright]无交集,

返回0

如果区间[left, right]被[qleft, qright]覆盖,

返回该区间的值

返回 左子树的访问值 + 右子树的访问值。



查询[2, 4]的和,

查询[0, 5]

查询[0, 2]

查询[0, 1] 返回0

查询[2, 2] 返回2

查询[3, 5]

查询[3, 4] 返回7

查询[5, 5] 返回0

最终结果为 $2 + 7 = 9$

例4:线段树的求和， 课堂练习

```
int sum_range_segment_tree(std::vector<int> &value,  
                           int pos, int left, int right,  
                           int qleft, int qright){  
    if ( 1 ) {  
        return 0;  
    }  
    if ( 2 ) {  
        3  
    }  
    int mid = (left + right) / 2;  
    return sum_range_segment_tree(value, pos * 2 + 1,  
                                   left, mid, qleft, qright) +  
        sum_range_segment_tree(value, pos * 2 + 2, mid + 1,  
                                   right, qleft, qright);  
}
```


例4:线段树的求和，实现

```
int sum_range_segment_tree(std::vector<int> &value,  
                           int pos, int left, int right,  
                           int qlong, int qright){  
    if ( qlong > right || qright < left ) {  
        return 0;  
    }  
    if ( qlong <= left && qright >= right ) {  
        return value[pos];  
    }  
    int mid = (left + right) / 2;  
    return sum_range_segment_tree(value, pos * 2 + 1,  
                                   left, mid, qlong, qright) +  
        sum_range_segment_tree(value, pos * 2 + 2, mid + 1,  
                                   right, qlong, qright);  
}
```

例4: 线段树的更新

传入参数:

线段树数组value(存储区间sum);

当前线段(节点)在线段树数组(value)中的下标pos;

当前线段的左端点left、右端点right,

待更新的数组位置下标index,

更新值new_value

如果当前访问的节点为线段树叶节点, 且为待更新数组下标index

更新该节点值为new_value

返回

计算中心点mid;

如果index <= mid:

递归更新该节点的左子树

否则

递归更新该节点的右子树

当前节点值为 左子树的访问值 + 右子树的访问值

更新原数组下标为2的值, 更新为10

搜索[0, 5]

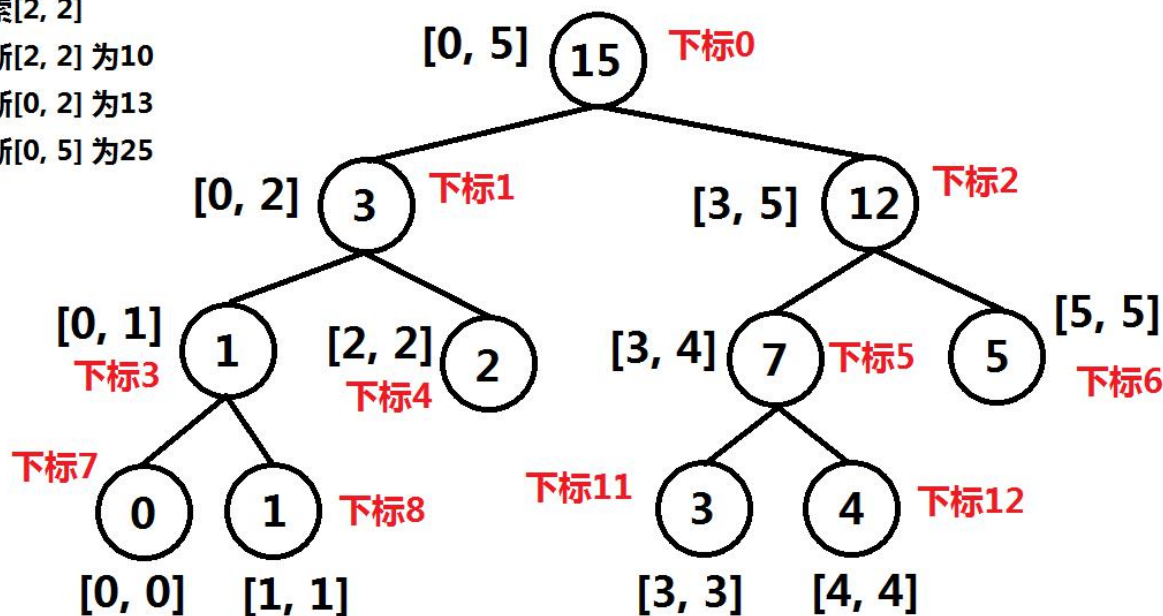
搜索[0, 2]

搜索[2, 2]

更新[2, 2] 为10

更新[0, 2] 为13

更新[0, 5] 为25



例4:线段树的更新, 课堂练习

```
void update_segment_tree(std::vector<int> &value,  
                        int pos, int left, int right,  
                        int index, int new value){  
    if ( 1 ) {  
        value[pos] = new value;  
        return;  
    }  
    int mid = (left + right) / 2;  
    if ( 2 ) {  
        update_segment_tree(value, pos * 2 + 1,  
                            left, mid, index, new value);  
    }  
    else {  
        update_segment_tree(value, pos * 2 + 2, mid + 1,  
                            right, index, new value);  
    }  
    value[pos] = 3  
}
```

例4:线段树的更新, 实现

```
void update_segment_tree(std::vector<int> &value,  
                        int pos, int left, int right,  
                        int index, int new value){  
    if (left == right && left == index){  
        value[pos] = new value;  
        return;  
    }  
    int mid = (left + right) / 2;  
    if (index <= mid){  
        update_segment_tree(value, pos * 2 + 1,  
                            left, mid, index, new value);  
    }  
    else{  
        update_segment_tree(value, pos * 2 + 2, mid + 1,  
                            right, index, new value);  
    }  
    value[pos] = value[pos * 2 + 1] + value[pos * 2 + 2];  
}
```


例4:线段树整体测试

```
int main() {
    std::vector<int> nums;
    for (int i = 0; i < 6; i++) {
        nums.push_back(i);
    }
    std::vector<int> value;
    for (int i = 0; i < 24; i++) {
        value.push_back(0);
    }
    build_segment_tree(value, nums, 0, 0, nums.size() - 1);
    printf("segment_tree:\n");
    print_segment_tree(value, 0, 0, nums.size() - 1, 0);
    int sum_range = sum_range_segment_tree(
        value, 0, 0, nums.size() - 1, 2, 4);
    printf("sum_range [2, 5] = %d\n", sum_range);
    update_segment_tree(value, 0, 0, nums.size() - 1, 2, 10);
    printf("segment_tree:\n");
    print_segment_tree(value, 0, 0, nums.size() - 1, 0);
    return 0;
}
```

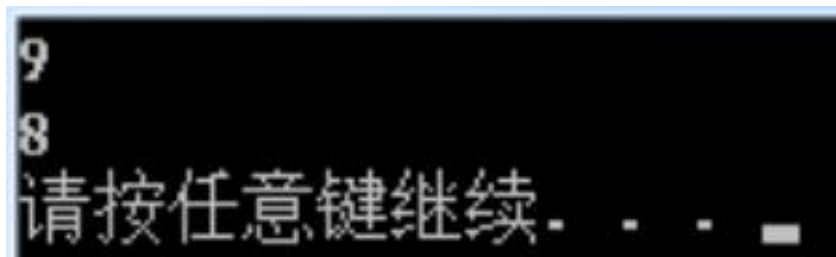
```
segment_tree:
[0 5][0] : 15
---[0 2][1] : 3
-----[0 1][3] : 1
-----[0 0][7] : 0
-----[1 1][8] : 1
-----[2 2][4] : 2
---[3 5][2] : 12
-----[3 4][5] : 7
-----[3 3][11] : 3
-----[4 4][12] : 4
-----[5 5][6] : 5
sum_range [2, 5] = 9
segment_tree:
[0 5][0] : 23
---[0 2][1] : 11
-----[0 1][3] : 1
-----[0 0][7] : 0
-----[1 1][8] : 1
-----[2 2][4] : 10
---[3 5][2] : 12
-----[3 4][5] : 7
-----[3 3][11] : 3
-----[4 4][12] : 4
-----[5 5][6] : 5
请按任意键继续. . .
```


例4:整体功能代码

```
class NumArray {
public:
    NumArray(std::vector<int> nums) {
        if (nums.size() == 0) { //注意数组为空的情况
            return;
        }
        int n = nums.size() * 4; //一般线段树数组大小是原数组大小长度的4倍
        for (int i = 0; i < n; i++) {
            _value.push_back(0);
        }
        build_segment_tree(_value, nums, 0, 0, nums.size() - 1);
        _right_end = nums.size() - 1; //线段的右端点，即为数组长度-1
    }
    void update(int i, int val) {
        update_segment_tree(_value, 0, 0, _right_end, i, val);
    }
    int sumRange(int i, int j) {
        return sum_range_segment_tree(_value, 0, 0, _right_end, i, j);
    }
private:
    std::vector<int> _value;
    int _right_end;
};
```

例4:测试与leetcode提交结果

```
int main() {  
    std::vector<int> nums;  
    nums.push_back(1);  
    nums.push_back(3);  
    nums.push_back(5);  
    NumArray num_array(nums);  
    printf("%d\n", num_array.sumRange(0, 2));  
    num_array.update(1, 2);  
    printf("%d\n", num_array.sumRange(0, 2));  
    return 0;  
}
```



```
9  
8  
请按任意键继续. . .
```

Range Sum Query - Mutable

Submission Details

10 / 10 test cases passed.

Status: **Accepted**

Runtime: 116 ms

Submitted: 0 minutes ago

结束

非常感谢大家！

林沐

疑问

□ 问题答疑：<http://www.xxwenda.com/>

■ 可邀请老师或者其他人回复问题

联系我们

小象学院：互联网新技术在线教育领航者

- 微信公众号：小象
- 新浪微博：ChinaHadoop

